Creative Lead.

From zero to a demo that wins, in five days.

"The voice of the user. The soul of the demo."

A complete, paste-ready Antigravity build guide for the Product & Urdu track of the Bulao project — AI Seekho 2026.

Every prompt is self-contained. Every file path, dependency, schema, and convention is spelled out. Antigravity will not need to guess.

How to use this document:
1. Read 'Your role' and 'System context' first (sections 01-02).
2. On each build day, paste the day's prompt into Antigravity.
3. Verify with the day's checklist before moving on.
4. Reference the appendices when Antigravity asks for clarification.

01 · YOUR ROLE

You own product & urdu.

You are the **Creative Lead** on the Bulao build team. There are four teammates total — Product & Urdu, Mobile, Backend, Infrastructure & Trace. This document is yours alone. Each teammate has their own pack. Do not try to do the others' work; coordinate handoffs through WhatsApp + a shared Drive.

What you own

- **The 50-example Urdu / Roman Urdu intent dataset**
- The Intent Agent prompt — iterated to 92%+ accuracy
- The Ranking Agent's Urdu reasoning quality
- The demo video script + voiceover direction
- The README's narrative + pitch sections
- The submission package final review

What you do *not* own

- Flutter or Python code (Mobile + Backend handle these)
- Cloud Run deployment (Infra handles)
- Antigravity screenshots & Plan Artifacts (Infra handles)

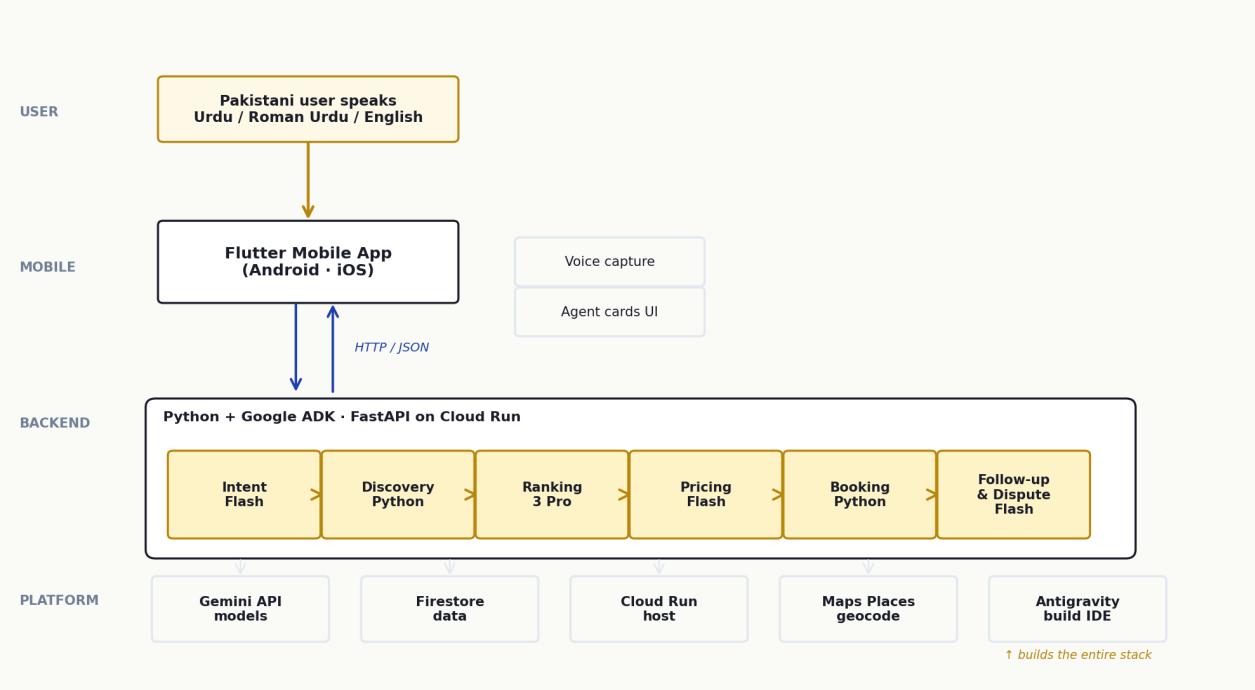
> *If you find yourself doing work outside the 'what you own' list, stop and ask in the team WhatsApp. Five days is short. Stay in your lane and move fast.*

02 · SYSTEM CONTEXT

What Bulao is, end to end.

Bulao is a voice-first, Urdu-native AI service orchestrator for Pakistan's informal economy — plumbers, electricians, AC technicians, beauticians, tutors. A user presses a big button, speaks in Urdu / Roman Urdu / English, and within ninety seconds watches **six AI agents** extract their intent, find providers, rank them, produce a transparent price quote, book a slot, and follow up through completion — including fair dispute resolution when something goes wrong.

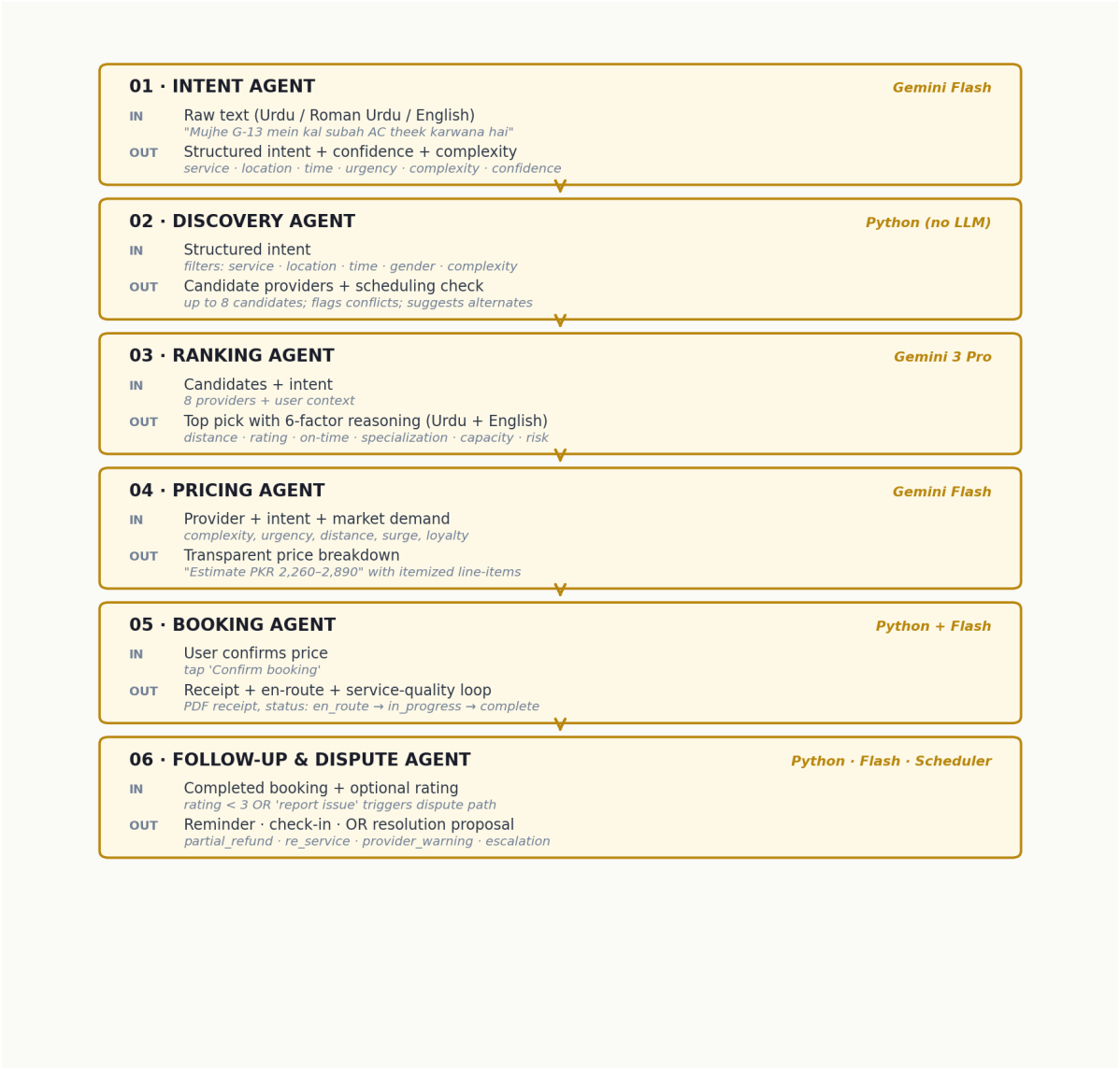
We are building this for the AI Seekho 2026 Antigravity Hackathon (Challenge 2). Five days, four teammates, one demo on May 20. Regional finals May 25-26, finals June 7 in Islamabad. Prize pool PKR 2.5M.



Three layers: a Flutter mobile app captures voice and renders six AgentCards; a Python backend on Cloud Run runs the six-agent pipeline using Google ADK and Gemini; Antigravity is the IDE that builds the stack — not part of the runtime, but the source of the 25%-weighted Agent Trace deliverable.

The six agents

Every teammate must know what each agent does, what it takes in, and what it produces. This is the shared mental model.



Your relationship to the six agents

You are the steward of the agents' **language quality**. The Intent Agent must understand Pakistani Urdu the way a Pakistani would. The Ranking Agent must explain its choices in natural Urdu, not translated English. The Pricing Agent must justify its price in language that feels fair, not corporate. The Booking Agent's confirmation must sound like a thoughtful friend, not an enterprise SMS. Every agent's behavior is governed by its system prompt — you own those prompts. You iterate them against the 50-example dataset until accuracy and tone are both right.

Key handoffs you must respect

FROM	Day	TO	Deliverable
You	1 evening	Backend	intent_examples.jsonl — at least 25 of 50 examples
You	2 evening	Backend	Full 50-example dataset + Intent Agent system prompt v1
You	3 evening	Backend	Ranking Agent system prompt v1 (Urdu reasoning style)
You	4 evening	Backend	Pricing + Booking + Follow-up prompts
You	5 morning	Infra	Demo voiceover script (Urdu + English)
You	5 afternoon	Infra	README narrative sections (Project Overview, How AG, Pitch)

03 · DAY 1 · BEFORE MAY 15

Day 0 (May 14) · setup

Before May 15, install Antigravity, grab your Gemini API key, clone the team repo, and read the official Challenge 2 spec twice. No Antigravity prompts today — these are manual steps you do at your desk.

Setup · install Antigravity and verify your tools

These are **not** Antigravity prompts — do them by hand on May 14. Antigravity cannot install software on your laptop for you. Burn no Day 1 hours on this; it costs the team their lead.

PROMPT · LOCAL ENVIRONMENT CHECKLIST (DO THIS YOURSELF)

Complete these on your laptop by end of Day 0 (May 14):

1. Install Google Antigravity (https://antigravity.google/download).
- Launch, sign in with a personal Gmail (the same one the team uses for GCP).
- Settings -> Agent panel:
Mode = Agent-assisted development
Artifact Review Policy = Asks for Review
Terminal Command Auto Execution = Request Review
Enable Terminal Sandbox = ON
Default model = Gemini 3 Pro
- Verify you can spawn a new Manager View workspace.

2. Get your Gemini API key from https://aistudio.google.com -> Get API key.
Create a key in a project named bulao-hackathon. Copy it; you will not
paste it anywhere in this role (Backend / Infra teammates handle that),
but you need it to validate prompts using AI Studio's playground.

3. Install a code editor that opens .md and .jsonl cleanly. VS Code is fine
(Antigravity is built on it). You will not write code, but you will read
what Antigravity writes.

4. Install Git. You will commit your work to the team repo:
git --version # any recent version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

5. Clone the team repo. The Infra teammate will share the URL by end of Day 0.
git clone
cd bulao
ls # you should see mobile/ backend/ docs/ at minimum

6. Read the official Challenge 2 specification (sent by the organizers).
Read the Bulao Engineering Playbook v2.0 (the companion document).
Open both in your browser as tabs you keep open all week.

> 7. Test that you can recall and recite Bulao's tagline in Urdu without thinking:
> "Bolo, aur kaam ho jaye."
> You will repeat this 20 times by Day 5. Start now.
>
> If anything fails, post in WhatsApp BEFORE Day 1 starts. Do not burn Day 1 hours on environment setup.

End-of-day verification

- Antigravity launches and you can spawn a new Manager View workspace.
- You have your Gemini API key saved in a password manager.
- The team repo is cloned at ~/bulao/ and backend/ exists.
- You have read the official Challenge 2 spec end-to-end (twice).
- You have read the Engineering Playbook v2.0 end-to-end (once).
- You can write down Bulao's six agents in order without looking.

04 · DAY 2 · MAY 15

Day 1 · scaffold + dataset anchors

Today you create the six empty agent prompt files (so Backend can wire to them tonight) and you hand-write the first 15 anchor examples for the Urdu dataset. These 15 anchors set the dialect that the next 35 will follow.

Morning · scaffold the six agent prompt files

Day 1's first task is creating the six empty files where each agent's system prompt will live. You will not write the prompts yet — just the files, with a placeholder header that names the agent and its purpose. The Backend teammate needs these files to exist so their llm_agent.py can load them on Day 1 evening.

PROMPT · CREATE THE SIX AGENT PROMPT FILES (ANTIGRAVITY)

Create the six agent prompt files at backend/agents/prompts/. Each file is markdown (.md). They will be loaded by the Backend teammate's code at module import time. You are creating the SCAFFOLDS only — the real system prompts will be pasted in later this week from Appendix A of this Role Pack.

File 1: backend/agents/prompts/intent_agent_system.md
Header line: # Intent Agent — system prompt (v0 scaffold)
Body placeholder:
> Extracts structured booking info (service_type, location, time_window, urgency,
> job_complexity, specialization_hint, gender_preference, confidence,
> clarification_question) from Urdu / Roman Urdu / English / code-switched input.
>
> PLACEHOLDER — to be replaced with v1 prompt on Day 2 (see Role Pack Appendix A).

File 2: backend/agents/prompts/discovery_agent_contract.md
Header line: # Discovery Agent — behavior contract (no LLM)
Body placeholder:
> This agent does not use an LLM. It is plain Python filtering, scheduling-
> conflict detection, and alternate-slot suggestion. See Role Pack Appendix A
> for the full contract.
>
> PLACEHOLDER — to be replaced on Day 4.

File 3: backend/agents/prompts/ranking_agent_system.md
Header line: # Ranking Agent — system prompt (v0 scaffold)
Body placeholder:
> Scores up to 8 provider candidates across six factors (distance, availability,
> rating, on-time, specialization, risk) using urgency-weighted profiles.
> Returns recommended_id + factor_scores + reasoning in Urdu and English.
>
> PLACEHOLDER — to be replaced on Day 3.

File 4: backend/agents/prompts/pricing_agent_system.md
Header line: # Pricing Agent — system prompt (v0 scaffold)
Body placeholder:
> Writes the natural-language explanation for a transparent line-item price
> quote. Inputs: intent, provider, market_demand, is_first_booking, line_items.
> Outputs: explanation_english, explanation_urdu, fairness_note.
>
> PLACEHOLDER — to be replaced on Day 3.

File 5: backend/agents/prompts/booking_agent_system.md
Header line: # Booking Agent — system prompt (v0 scaffold)
Body placeholder:
> Writes the WhatsApp-style confirmation message in Urdu and English.
> Inputs: booking, provider, user_name. Outputs: english + urdu strings.
>
> PLACEHOLDER — to be replaced on Day 4.

File 6: backend/agents/prompts/followup_dispute_agent_system.md
Header line: # Follow-up & Dispute Agent — system prompt (v0 scaffold)
Body placeholder:
> Two modes. Mode 1 = reminder/check-in messages. Mode 2 = dispute
> classification + resolution proposal.
>
> PLACEHOLDER — to be replaced on Day 4.

After creating all six files:
- Stage them with `git add backend/agents/prompts/*.md`.
- Commit with message: `chore(prompts): scaffold v0 placeholder system prompts`.
- Do not push yet; we will push together with the dataset commit later today.

Confirm by listing the directory: `ls -la backend/agents/prompts/`. You should see exactly six .md files.

Late morning · start the 50-example dataset

The 50-example utterance dataset (backend/data/intent_examples.jsonl) is the single most important artifact you own. It is the regression suite for the Intent Agent and the source of truth for what 'natural Pakistani input' looks like. Today you anchor it with 15 hand-written examples; Antigravity extends to 50 tomorrow. **You write the first 15 yourself** — do not let Antigravity generate the anchors. Antigravity is great at extending a pattern but bad at originating one in a specific dialect.

PROMPT · WRITE THE 15 ANCHOR EXAMPLES (DO THIS YOURSELF, THEN ASK AG TO VALIDATE)

Open backend/data/intent_examples.jsonl in your editor. The file is JSON Lines — one JSON object per line, no trailing comma, no wrapping array.

Copy the 15 anchor examples from Appendix B of this Role Pack VERBATIM into the file. These are hand-curated to cover the dialect range. Do not paraphrase them.

After pasting all 15, run this Antigravity prompt to validate the file:

> Open backend/data/intent_examples.jsonl. For each line, validate that:
> (1) it parses as valid JSON,
> (2) it has both `input` (string) and `expected` (object) keys,
> (3) `expected` contains at minimum: service_type, time_window, urgency, job_complexity, confidence_min,
> (4) `expected.service_type` is one of: plumber, electrician, ac_technician, geyser_technician, carpenter, painter, beautician, tutor, appliance_repair, gas_leak_specialist.
> Report any lines that fail validation with the line number and the specific problem. Do NOT modify the file — just report.

Fix any reported issues by hand. Commit:
- git add backend/data/intent_examples.jsonl
- git commit -m "data: 15 anchor examples for Urdu intent dataset"
- git push

End-of-day verification

- backend/agents/prompts/ has exactly six .md files.
- Each file has the correct header line and PLACEHOLDER body.
- backend/data/intent_examples.jsonl has exactly 15 lines.
- Every line parses as valid JSON.
- Every line has both input and expected keys.
- Coverage: at least 3 emergency, 3 high-urgency, 3 with confidence<0.7.
- Coverage: at least 5 Urdu/Roman Urdu, 3 English, 7 code-switched.
- Both commits pushed; the Backend teammate can pull them.

> *By 6pm tonight, push the prompt scaffolds and the 15 anchor examples. Backend will pull and verify their code can load all six .md files at module import.*

05 · DAY 3 · MAY 16

Day 2 · full dataset + Intent Agent v1 + eval loop

By 6pm tonight, the Intent Agent passes 92%+ on service_type and 85%+ on job_complexity across all 50 examples. The eval-iterate loop is the heart of today; expect 3-5 iteration rounds.

Morning · extend the dataset from 15 to 50 examples

Today Antigravity extends your 15 anchors to a full 50, following the dialect and coverage patterns you established. You will review the 35 new examples carefully — Antigravity will occasionally invent un-Pakistani phrasing (too formal Urdu, English idioms translated literally, neighborhood names from India). You catch these. Your eye is the moat.

PROMPT · EXTEND INTENT_EXAMPLES.JSONL TO 50 (ANTIGRAVITY)

Read backend/data/intent_examples.jsonl. It currently has 15 hand-written anchor examples. Extend it to exactly 50 lines total by adding 35 new examples. Match the dialect, structure, and tone of the anchors precisely.

REQUIREMENTS FOR THE 35 NEW EXAMPLES:

Language mix (across the 35):
- 15 in Urdu / Roman Urdu (the way Pakistanis text)
- 7 in English with a Pakistani register ("Need an electrician for tomorrow morning")
- 13 code-switched (e.g., "Kal morning AC service chahiye")

Coverage targets (across all 50, including the 15 anchors):
- All 10 service categories represented at least 3x each: plumber, electrician, ac_technician, geyser_technician, carpenter, painter, beautician, tutor, appliance_repair, gas_leak_specialist
- At least 8 different neighborhoods (mix of Islamabad sectors like F-6/F-10/G-13, Rawalpindi, Bahria Town, DHA Karachi, DHA Lahore)
- Urgency distribution: 8 emergency, 25 normal, 17 flexible
- Complexity distribution: ~18 basic, ~22 intermediate, ~10 complex
- 8 ambiguous examples (confidence_min < 0.7, needs_clarification = true)
- 5 mid-sentence corrections ("Mujhe plumber... actually electrician chahiye")
- 5 multi-service ("plumber AND electrician chahiye for kitchen renovation")

DIALECT RULES (critical):
- Use natural Pakistani Roman Urdu spelling, not Indian/Hindi:
YES: "karwana", "chahiye", "ghar", "kal subah"
NO: "karwaana", "chaahiye", "ghara"
- Common verbs: chahiye (need), karwana (get done), bhejo (send), theek karwana (repair)
- Time markers: kal subah, kal sham, abhi, jaldi, kabhi bhi
- Avoid words a Pakistani Urdu speaker would not use in casual texting (e.g. "vidyut" for electricity).
- Pakistani neighborhood naming conventions: "G-13", "F-10/3" (with slash for sub-sector), "Bahria Phase 4", "DHA Phase 6 Karachi".

JSON FORMAT (one per line, no trailing comma, no wrapping array):
{"input": "", "expected": {"service_type": "", "location": "", "city": "Islamabad" | "Rawalpindi" | "Karachi" | "Lahore", "time_window": "", "urgency": "", "job_complexity": "", "specialization_hint": "", "gender_preference": "", "confidence_min": , "needs_clarification": , "raw_notes": ""}}

For ambiguous examples, ALSO include "expected_clarification_topic": "".

CONSTRAINTS:
- 50 unique inputs. No duplicates, no trivial variations of the same sentence.
- The file must parse as valid JSON Lines (every line is independently valid JSON).
- Lines should be lined up so that any reviewer can read three random lines and pattern-match the rest.

DO NOT remove or modify any of the 15 anchor examples. Append your 35 new ones below them.

After generating, save the file and confirm it now has exactly 50 lines (use `wc -l backend/data/intent_examples.jsonl`).

Late morning · write the Intent Agent system prompt v1

With the dataset complete you can now write the system prompt that will be evaluated against it. The full text of v1 lives in Appendix A.1 of this Role Pack. You will paste it into the file, then run an eval to see how well it performs. Expect 80-85%% accuracy on first run; you will iterate this afternoon to reach 90%%+.

PROMPT · REPLACE INTENT_AGENT_SYSTEM.MD WITH V1 PROMPT (ANTIGRAVITY)

Replace the contents of backend/agents/prompts/intent_agent_system.md entirely with the v1 system prompt provided below. Preserve no PLACEHOLDER text.

The full v1 prompt to paste is provided in Appendix A.1 of the Product & Urdu Role Pack. Open that PDF, find Appendix A.1, and copy the prompt VERBATIM into the file. Do not paraphrase or "improve" the prompt — it has been engineered to balance accuracy, latency, and Urdu naturalness.

After pasting, the file should:
- Start with the line "You are the Intent Agent for Bulao..."
- End with "...the orchestrator will surface it to the user."
- Have no markdown code fences around the content (it is the prompt itself, not a code listing).
- Be approximately 2000 words.

Save the file. Commit:
- git add backend/agents/prompts/intent_agent_system.md
- git commit -m "agents(intent): v1 system prompt with confidence + complexity fields"
- git push

Afternoon · first eval and iteration loop

The Backend teammate's eval harness at backend/scripts/eval_intent.py is ready by now (it ran successfully on the 15 anchors yesterday evening). You run it against the full 50, study the confusion matrix, and iterate the system prompt until service_type accuracy is ≥92%% and job_complexity accuracy is ≥85%%.

PROMPT · RUN THE INTENT AGENT EVAL AND REPORT (ANTIGRAVITY)

Run the Intent Agent evaluation harness against the full 50-example dataset and produce a report.

1. From the repo root, run:
cd backend
python scripts/eval_intent.py --dataset data/intent_examples.jsonl --report eval_report.md

2. Read backend/eval_report.md. It contains:
- Overall service_type accuracy (%)
- Overall job_complexity accuracy (%)
- Confidence calibration: for examples with confidence_min < 0.7, did the model also return confidence < 0.7? Vice versa?
- Confusion matrix on service_type (which categories get confused?)
- List of every failing example with the model's output vs expected.

3. Print a summary in the chat: two accuracy percentages, the top 3 confused category pairs, and the top 5 failing examples.

4. For each of the top 5 failing examples, propose a one-sentence diagnosis. Common patterns:
- "Model returned plumber but expected ac_technician — the word 'leak' in the input ambiguates."
- "Model returned confidence 0.92 but expected <0.7 — input is ambiguous on location."
- "Model returned complex but expected intermediate — model overweights the word 'inverter'."

5. Stop after the report. DO NOT modify intent_agent_system.md in this prompt. The iteration prompt is the next step.

Late afternoon · refine the prompt and re-eval

Based on the eval report, you will give Antigravity targeted refinements to make to the system prompt. Each refinement should address ONE failure mode at a time. After each change, re-run the eval. Target: 92%% service_type, 85%% complexity by 6pm. If you hit it earlier, stop — do not over-tune.

PROMPT · TARGETED PROMPT REFINEMENT (ANTIGRAVITY, ONE ROUND)

Apply ONE targeted refinement to backend/agents/prompts/intent_agent_system.md based on the failure mode I describe below. Then re-run the eval and report the new accuracy delta.

FAILURE MODE TO ADDRESS (paste from your eval analysis):

INSTRUCTIONS:
1. Read intent_agent_system.md.
2. Identify where in the prompt this failure mode is (or is not) addressed.
3. Add ONE paragraph or ONE bullet to the prompt that disambiguates the specific case.
4. Keep the addition under 60 words.
5. Do not remove or rewrite any existing content.
6. Re-run the eval: cd backend && python scripts/eval_intent.py --dataset data/intent_examples.jsonl --report eval_report.md
7. Compare new accuracy to the previous report. Print: "Service_type before: X%, after: Y%. Complexity before: X%, after: Y%."

If accuracy DROPS, revert your change and report the failure.
If accuracy STAYS THE SAME, report and ask me whether to revert.
If accuracy IMPROVES, commit:
git commit -am "agents(intent): disambiguate "

> DO NOT make multiple changes in one round. We iterate one failure mode at a time so we can attribute every accuracy gain to a specific edit.

End-of-day verification

- backend/data/intent_examples.jsonl has exactly 50 valid JSON lines.
- Distribution checked: 8 emergency, 25 normal, 17 flexible.
- Distribution checked: 18 basic, 22 intermediate, 10 complex.
- 8 ambiguous examples with confidence_min < 0.7 and expected_clarification_topic.
- intent_agent_system.md is the v1 prompt (no PLACEHOLDER).
- Latest eval_report.md shows service_type accuracy ≥ 92%%.
- Latest eval_report.md shows job_complexity accuracy ≥ 85%%.
- Confidence calibration: low-confidence examples correctly flagged at ≥ 75%%.
- All commits pushed; Backend can pull and pin the prompt for the demo.

> *The 92/85 numbers go straight into the README's eval section. Save the eval_report.md from your FINAL run — that's what gets quoted in Section 8 of the README on Day 5.*

06 · DAY 4 · MAY 17

Day 3 · Ranking + Pricing agent prompts

Two agents come online today — Ranking and the new-in-v2 Pricing Agent. The Ranking Agent is the most user-visible LLM in the system, so Urdu naturalness matters as much as accuracy. The Pricing Agent's prompt is shorter but the transparency rules must be enforced exactly.

Morning · replace the Ranking Agent prompt with v1

The Ranking Agent is the most user-visible LLM in the system — its Urdu reasoning literally types out on screen during the demo. The v1 prompt in Appendix A.3 specifies the six-factor scoring algorithm and the language tone. Paste it in; Backend will wire it to the live providers data this afternoon.

PROMPT · REPLACE RANKING_AGENT_SYSTEM.MD WITH V1 (ANTIGRAVITY)

Replace the contents of backend/agents/prompts/ranking_agent_system.md entirely with the v1 system prompt provided in Appendix A.3 of the Product & Urdu Role Pack. Preserve no PLACEHOLDER text.

The full v1 prompt to paste is in the Role Pack PDF, Appendix A.3. Copy it verbatim. It is approximately 1400 words.

After pasting, the file should:
- Start with "You are the Ranking Agent for Bulao..."
- End with "...OUTPUT NOTHING EXCEPT THE JSON."
- Contain the explicit six factors, the weighting profiles by urgency, and the required JSON output shape including factor_scores.

Save the file. Commit:
- git add backend/agents/prompts/ranking_agent_system.md
- git commit -m "agents(ranking): v1 system prompt with six-factor scoring"
- git push

Late morning · test the Ranking Agent's Urdu output quality

Accuracy isn't the issue with the Ranking Agent — the rubric says the reasoning must reference specific numbers. The risk is the *Urdu* reading wooden, like translated English. You will test against 5 specific scenarios and grade the Urdu naturalness on a 1-5 scale. If average < 4, iterate.

PROMPT · TEST RANKING AGENT URDU NATURALNESS (ANTIGRAVITY)

Run the Ranking Agent against 5 hand-crafted scenarios and report the Urdu reasoning quality. The Backend teammate has wired the agent to a CLI by now (backend/scripts/run_ranking.py).

For each scenario:
1. Run: cd backend && python scripts/run_ranking.py --scenario
2. Capture the reasoning_urdu field.
3. Grade it on a 1-5 scale:
5 = sounds like a Pakistani writing a WhatsApp message
4 = correct Urdu but slightly formal
3 = readable but feels translated
2 = grammatical issues or English-leaning idioms
1 = unreadable / wrong dialect

THE 5 SCENARIOS (these are pre-loaded in backend/data/ranking_scenarios.json):

Scenario 1: Emergency plumber — leak at midnight
Expected: urgency=emergency, distance and availability dominate

Scenario 2: Normal AC service — picking between equally close options
Expected: rating and on-time should dominate

Scenario 3: Female beautician for a bridal party
Expected: specialization match (bridal) + gender preference

Scenario 4: Complex AC repair — inverter PCB
Expected: specialization (inverter_ac) + years_experience dominate

Scenario 5: Two providers tied on score
Expected: tradeoffs section is well-written, with concrete differences

After running all 5, print a 6-row table:
Scenario | Recommended | Urdu grade (1-5) | One-sentence comment

Then print the average Urdu grade.

If the average is < 4.0, identify the most common Urdu issue (e.g. "uses 'distance' when 'fasla' would be natural") and propose ONE targeted prompt edit in a follow-up message. Do not edit the prompt in this run — just report.

Afternoon · replace the Pricing Agent prompt with v1

The Pricing Agent is new in v2 — the rubric requires transparent pricing with line items. The prompt is short (about 700 words) because the math is deterministic Python; the LLM only writes the explanation. The full v1 is in Appendix A.4. After pasting, you will eyeball the Urdu output on 3 sample quotes to make sure 'surge' and 'multiplier' read naturally.

PROMPT · REPLACE PRICING_AGENT_SYSTEM.MD WITH V1 (ANTIGRAVITY)

Replace the contents of backend/agents/prompts/pricing_agent_system.md entirely with the v1 system prompt provided in Appendix A.4 of the Product & Urdu Role Pack.

The full v1 prompt to paste is in the Role Pack PDF, Appendix A.4. Copy it verbatim. It is approximately 700 words.

After pasting, the file should:
- Start with "You are the Pricing Agent for Bulao..."
- End with "...OUTPUT NOTHING EXCEPT THE JSON."
- List all 7 line item types in the specified order.
- Specify rules for when to mention surge, complexity, and loyalty.

Save. Then test on three pre-loaded scenarios:
cd backend && python scripts/run_pricing.py --scenario 1
... --scenario 2 (with surge)
... --scenario 3 (complex job, first booking)

For each, read the explanation_urdu. Confirm:
- Surge is mentioned ONLY in scenario 2.
- Complexity is mentioned ONLY in scenario 3.
- Welcome discount is mentioned ONLY in scenario 3 (first booking).
- The fairness_note specifies what the provider receives.

Report the three explanation_urdu strings and your verdict (pass/fail per scenario). If any fail, identify the rule violation and propose a targeted prompt edit in a follow-up.

Commit when satisfied:
- git commit -am "agents(pricing): v1 prompt + verified scenario 1/2/3 output"
- git push

End-of-day verification

- ranking_agent_system.md is v1 (no PLACEHOLDER).
- pricing_agent_system.md is v1 (no PLACEHOLDER).
- Ranking Agent Urdu grade averages ≥ 4.0 across the 5 test scenarios.
- Pricing Agent mentions surge ONLY when surge_factor > 1.0.
- Pricing Agent mentions complexity multiplier ONLY when complexity != basic.
- Pricing Agent fairness_note quantifies what the provider receives.
- All commits pushed; Backend can pin both prompts.

07 · DAY 5 · MAY 18

Day 4 · Discovery + Booking + Follow-up & Dispute + voiceover

All six agents at v1 by end of day. The agent quality sweep produces a written report that the team uses overnight to refine. The demo voiceover script ships tonight so tomorrow's shoot can run smoothly.

Morning · replace the Discovery Agent contract

The Discovery Agent doesn't use an LLM but its behavior contract is still your responsibility — you write the spec, Backend implements it. The v1 contract in Appendix A.2 documents the scheduling-intelligence path with 30-minute travel buffer, alternate-slot suggestions, and the no-match-reason fallback messages. Paste it in and let Backend implement against it today.

PROMPT · REPLACE DISCOVERY_AGENT_CONTRACT.MD WITH V1 (ANTIGRAVITY)

Replace the contents of backend/agents/prompts/discovery_agent_contract.md entirely with the v1 behavior contract provided in Appendix A.2 of the Product & Urdu Role Pack.

The contract describes:
- The 8-step algorithm (service filter, specialization, location, gender, complexity, time-window with travel buffer, scheduling intelligence, pre-rank).
- The DiscoveryResult Pydantic shape.
- The ProviderCandidate field list including v2 fields (on_time_score, cancellation_rate, etc.).
- Phone masking rules.
- Example no_match_reason strings in Urdu and English.

After pasting, the file should be approximately 800 words and read as a contract document, not a system prompt.

Commit:
- git commit -am "agents(discovery): v1 behavior contract"
- git push

NOTE: This file is consumed by humans (the Backend teammate) and by the future Antigravity prompt that implements the discovery_agent.py module. It is not loaded by any agent at runtime.

Late morning · replace Booking + Follow-up/Dispute prompts

Two prompts to paste this morning. The Booking Agent writes the confirmation message that goes into the receipt PDF and the WhatsApp-style chat bubble. The Follow-up & Dispute Agent has two modes — reminder/check-in on the happy path, and dispute classification + resolution when rating < 3. Both prompts are in Appendix A.5 and A.6.

PROMPT · REPLACE BOOKING + FOLLOWUP-DISPUTE PROMPTS (ANTIGRAVITY)

Replace two prompt files with v1 content from the Role Pack.

File 1: backend/agents/prompts/booking_agent_system.md
- Replace entirely with Appendix A.5 of the Role Pack.
- Approximately 500 words.
- Specifies: tone (warm, brief, confident), required content (greeting, booking_id, provider name + positive attribute, scheduled_time in natural Urdu, price range), JSON output with english + urdu keys.

File 2: backend/agents/prompts/followup_dispute_agent_system.md
- Replace entirely with Appendix A.6 of the Role Pack.
- Approximately 1200 words.
- Specifies BOTH modes with separate input schemas, classification options, resolution rules, and JSON output shapes.

After pasting both, verify:
- Booking prompt: ends with "OUTPUT FORMAT — JSON ONLY with keys "english" and "urdu". No preamble. No markdown."
- Follow-up/Dispute prompt: contains "MODE 1" and "MODE 2" section markers.
- Follow-up/Dispute prompt: lists 6 dispute classifications (no_show, quality_complaint, price_disagreement, overrun, damage, other).
- Follow-up/Dispute prompt: lists 5 resolution types (partial_refund, full_refund, re_service, provider_warning, escalate_to_human).

Commit both together:
- git commit -am "agents(booking+followup): v1 prompts incl. dispute classification"
- git push

Afternoon · end-to-end agent quality sweep

All six agents now have v1 prompts. Backend has wired them all. Today's afternoon job is a quality sweep: you run one full end-to-end pipeline three times with three different Urdu inputs and grade each agent's output 1-5 on naturalness. You file Urdu defects as one-line entries in a shared doc; Backend fixes the prompts overnight if needed.

PROMPT · RUN E2E AGENT QUALITY SWEEP (ANTIGRAVITY)

Run the full Bulao pipeline against three distinct Urdu inputs and grade every agent's output for Urdu naturalness and tone. Produce a one-page report.

THE THREE INPUTS:

Input A (normal, clear):
"Mujhe G-13 mein kal subah AC theek karwana hai, inverter wala hai"

Input B (emergency, ambiguous location):
"Abhi jaldi se plumber bhejo, geyser leak ho raha hai"

Input C (complex, with female preference):
"DHA Karachi mein bridal makeup ke liye female beautician chahiye, Friday ke liye"

FOR EACH INPUT:

1. Run: cd backend && python scripts/run_pipeline.py --input "" --output reports/sweep_.json
2. Open the resulting JSON and read each of the 6 agents' outputs.
3. For agents that produce Urdu text (Intent's clarification_question, Ranking's reasoning_urdu, Pricing's explanation_urdu, Booking's urdu, Follow-up's urdu), grade 1-5:
5 = sounds like a Pakistani's WhatsApp message
4 = correct but slightly formal
3 = readable but feels translated
2 = grammar issues or English idioms
1 = unreadable

4. Note ANY specific defect (a wrong word, an unnatural phrase, missing context).

THE REPORT:

Produce a markdown report at reports/agent_quality_sweep.md with this structure:

Agent Quality Sweep — Day 4 Afternoon

Summary
Average Urdu grade: X.X / 5.0
Number of defects flagged: N

Per-input breakdown
Input A: ...
Agent	Urdu grade	Defects
Intent	5	none
Ranking	4	"fasla" said as "distance"
Pricing	5	none
Booking	4	"tajurba" repeated twice
Follow-up	5	none

Input B: ...
(same table)

Input C: ...
(same table)

Recommended prompt edits
For each agent with average < 4.5, list ONE specific edit (max 30 words each).

After producing the report, post it in the team WhatsApp. DO NOT edit prompts in this run — just diagnose. Iteration happens in a follow-up prompt.

Evening · demo voiceover script (Urdu + English)

Tomorrow afternoon you direct the demo video shoot. Tonight you write the voiceover script — 4:30 of screen time, Urdu primary with English subtitles. You will read this script over the recorded screen capture. The shot list lives in Appendix C; you write the words that go over each shot.

PROMPT · DRAFT THE DEMO VOICEOVER SCRIPT (ANTIGRAVITY)

Read Appendix C of the Product & Urdu Role Pack — the demo storyboard with 11 timestamped shots from 0:00 to 4:30. Produce a complete voiceover script at docs/demo-voiceover.md.

FORMAT:

The file should have one section per shot. Each section:

0:00–0:15 — opening
Visual: [paraphrase from storyboard]
Voiceover (Urdu): ""
Subtitle (English): ""

REQUIREMENTS:

1. Total runtime when read at conversational pace: 4:00-4:30 (slightly under the 4:30 storyboard so you have breathing room).

2. Urdu lines must be Roman Urdu, read like spoken Urdu, not formal Urdu. Use "aapka" not "aap ka", "kya" not "kiya", "kareingay" not "karenge".

3. English subtitles must be naturally translated, NOT literal. "Bolo, aur kaam ho jaye" -> "Speak, and it's handled" (NOT "Speak, and work gets done").

4. The opening (0:00) and closing (4:10) shots must include the tagline: "Bolo, aur kaam ho jaye."

5. Specific lines that MUST appear (these are the rubric-winning moments):
- When the Intent Agent reveals confidence: "Pehla agent samajhta hai kya kaam hai, kab chahiye, kitna complex hai."
- When the Ranking Agent reveals reasoning: "Sab nahi, sirf ek best provider — aur kyun, woh bhi bata raha hai."
- When the Pricing Agent reveals line items: "Har rupay ka hisaab — kuch chupaya nahi."
- When the Dispute Agent resolves: "Aakhri agent dispute ko insaaf ke saath handle karta hai."

6. No filler. Every line must add information or advance the story.

After drafting, count word density per 15-second shot. If any shot exceeds 35 words of voiceover, trim.

Commit the file:
- git add docs/demo-voiceover.md
- git commit -m "demo: voiceover script v1 (Urdu + English)"
- git push

End-of-day verification

- Discovery Agent contract document is v1 (no PLACEHOLDER).
- Booking Agent prompt is v1.
- Follow-up & Dispute Agent prompt is v1 with both MODE 1 and MODE 2.
- Agent quality sweep report exists at reports/agent_quality_sweep.md.
- Average Urdu grade across the 3 inputs × 5 agents is ≥ 4.2 / 5.0.
- Demo voiceover script is committed at docs/demo-voiceover.md.
- All six required mandatory lines are present in the voiceover.
- Total voiceover runtime when read at pace is 4:00–4:30.

> *Push the demo voiceover script BEFORE midnight. The Infra teammate will load it into their video edit timeline first thing tomorrow morning.*

08 · DAY 6 · MAY 19

Day 5 · README + demo direction + pitch + submit

Submission day. Generate the README from spec, generate the closing card, direct the demo shoot (in person), draft the pitch script, run the final-review checklist, sign off for Infra to submit.

Morning · produce the README

The README is what judges see when they click the GitHub link. It is the only document many judges will actually read. Today you generate it from a structured spec (Appendix D) and review every section for tone, accuracy, and visual hierarchy. Done well it takes the documentation/polish points uncontested.

PROMPT · GENERATE README.MD FROM SPEC (ANTIGRAVITY)

Create README.md at the repo root from the structured specification provided in Appendix D of the Product & Urdu Role Pack. The README has exactly 11 top-level sections, in this order:

1. Project Overview (200 words)
2. Demo Video (large embed of YouTube unlisted link)
3. The 6-Agent Architecture (diagram + brief description per agent)
4. How Antigravity Was Used (4 screenshots, 200 words each)
5. Tools & APIs (bulleted list)
6. Setup & Running Locally (step-by-step terminal commands)
7. Deployment (gcloud commands)
8. The 50-Example Urdu Dataset (link + 5 example rows + accuracy summary)
9. Roadmap / What We'd Build Next (5 bullets)
10. Team (names, roles, photos)
11. Acknowledgements (InnoVista, MoITT, Telenor, Google)

CONSTRAINTS:

- Total length 1500-2500 words.
- Embed images using relative paths (./docs/diagrams/architecture.png).
- The architecture diagram from docs/diagrams/architecture.png is large, full-width, near the top.
- Code blocks for setup commands use triple-backtick bash fencing.
- "How Antigravity Was Used" must include 4 screenshots from docs/antigravity-trace/ with captions.
- Section 8 must show the actual eval accuracy numbers from backend/eval_report.md.
- Section 10 has team names but no personal details beyond first name + role.

OPEN APPENDIX D OF THE ROLE PACK FOR THE EXACT CONTENT TO WRITE PER SECTION. Do not improvise content for sections 1, 3, 4 — those have specific text I have written for you. Sections 5, 6, 7 are generated from the actual repo state.

After generating, render the README in a preview and confirm:
- All 11 sections present in the right order.
- Architecture diagram appears full-width near top.
- 4 Antigravity screenshots embedded with captions.
- Eval accuracy numbers match backend/eval_report.md.
- No "TODO" or "PLACEHOLDER" text anywhere.

Commit:
- git add README.md
- git commit -m "docs(readme): production README for AI Seekho 2026 submission"
- git push

Late morning · generate the demo closing card

The demo video ends on a 6-second closing card with the Bulao wordmark, the tagline, the 6-agent diagram, and the team credits. Antigravity can produce this as a standalone MP4 using OpenCV. The output goes into the Infra teammate's video edit timeline.

PROMPT · GENERATE DEMO CLOSING CARD MP4 (ANTIGRAVITY)

Generate a 6-second 1080x1920 portrait MP4 file at docs/demo-assets/closing_card.mp4 for the Bulao demo video closing.

SHOT BREAKDOWN (60fps, 360 frames total):

Frames 0-30 (0.0s-0.5s): black screen
Frames 30-90 (0.5s-1.5s): Bulao wordmark fades in
- Text "Bulao." in Helvetica Bold 240pt
- Color: #b8860b (gold) on #1a1d29 (ink) background
- Centered, slight Y-offset above center
Frames 90-180 (1.5s-3.0s): tagline types out below wordmark
- Text "Bolo, aur kaam ho jaye." in Helvetica 60pt
- Color: #e8e8e8 (off-white)
- Typewriter effect: 1 character per 3 frames
Frames 180-300 (3.0s-5.0s): 6-agent flow diagram slides in below
- Use docs/diagrams/agent_flow.png (the 6-box vertical flow)
- Slide up from bottom, ease-out
- Final height 700px, centered horizontally
Frames 300-360 (5.0s-6.0s): team credits below diagram
- "Team: Arslan · [member 2] · [member 3] · [member 4] · [member 5]"
- Helvetica 40pt
- Color: #b8860b
- Hold for 1 full second before video ends

USE OpenCV (cv2) + PIL for text rendering. NO external dependencies beyond opencv-python, pillow, numpy.

OUTPUT REQUIREMENTS:
- File path: docs/demo-assets/closing_card.mp4
- Format: H.264, mp4 container
- Resolution: 1080x1920 (portrait, phone aspect)
- Frame rate: 60fps
- Duration: 6.0 seconds (360 frames)
- File size: < 5MB

After generating, play it back and confirm:
- Black-to-fade-in is smooth.
- The Bulao wordmark is sharp at 240pt.
- The tagline types out, character-by-character.
- The 6-agent diagram slides in smoothly.
- Credits appear and hold for 1 second.

If the team names are not yet known, leave a placeholder "[member name]" and generate a final pass at end of day. Commit:
- git add docs/demo-assets/closing_card.mp4
- git commit -m "demo: closing card 6s MP4 for video edit timeline"
- git push

Afternoon · direct the demo video shoot

This is a non-Antigravity hour. You and the Mobile teammate sit together with the demo phone. You read the voiceover script aloud while they perform the interactions. Three takes per shot, then the Infra teammate edits tonight.

PROMPT · DEMO SHOOT CHECKLIST (DO THIS IN PERSON, NOT IN ANTIGRAVITY)

The demo video shoot. Do this in person, not via Antigravity.

PRE-SHOOT (15 minutes):
- Demo phone: fully charged, on Do Not Disturb, brightness at 80%%.
- Cloud Run service: min-instances bumped from 0 to 1 (Infra does this) so cold starts don't sabotage the demo.
- Wi-Fi: stable, but airplane mode + the team's phone hotspot is the safer call.
- The /orchestrate endpoint: tested with the canonical demo input two minutes before recording.
- Tripod or steady surface. Phone in landscape OR portrait — match the closing_card.mp4 (portrait).
- Quiet room. No ceiling fan. No traffic noise outside.
- Backup phone running OBS or QuickTime, recording the demo phone's screen as a fallback.

DURING THE SHOOT — eleven takes (one per storyboard beat in Appendix C):

For each take:
1. Mobile teammate performs the on-screen action.
2. You read the voiceover script in Urdu, naturally — not too slow, not announcer.
3. Three clean takes per beat. The Infra teammate will pick the best in edit.
4. Between takes, do not touch the phone — keep it on the same screen until the next take's setup.

THE ELEVEN BEATS (timestamps from docs/demo-voiceover.md):
0:00–0:15 opening — black screen + tagline
0:15–0:30 press-and-hold mic button + speak
0:30–0:40 intent extracted
0:40–1:00 Intent Agent card with confidence + complexity reveal
1:00–1:30 Discovery Agent + map slides in
1:30–2:00 Ranking Agent reasoning types out
2:00–2:30 Pricing Card reveals line items
2:30–3:00 Booking confirm + receipt slides up
3:00–3:20 Reminder notification fires
3:20–3:50 Dispute path — 2 stars triggers DisputeScreen + resolution
3:50–4:30 closing — clean reminder + closing card

POST-SHOOT:
- AirDrop / Send all takes to Infra teammate.
- Confirm Infra has docs/demo-voiceover.md and docs/demo-assets/closing_card.mp4.
- You are free until the README final review.

EMERGENCY: If voice input fails on the demo phone DURING the shoot, switch to text-input mode (the app supports it via the keyboard icon). The voiceover script does not need to change. Do not abandon the shoot; the text-input path is acceptable and the rubric does not require live voice.

Evening · pitch script for regional finals (May 25-26)

Submission is tonight; regional finals are 5 days away. You will pitch live in Islamabad on June 7. Write the pitch script now while the project is fresh in your head. The script is in Appendix E; this prompt produces the version tailored to your team's actual demo content.

PROMPT · GENERATE THE PITCH SCRIPT (ANTIGRAVITY)

Generate the live-pitch script at docs/pitch-script.md based on the template in Appendix E of the Product & Urdu Role Pack. Tailor with the team's actual results and names.

THE PITCH HAS FOUR PARTS (8 minutes total — 3-minute pitch + 5-minute Q&A; from judges):

PART 1 — Hook (0:00-1:00, ~150 words):
- Open with the Pakistani service-discovery pain point told as a story.
- Specific example: someone trying to find an AC technician through WhatsApp forwards in May heat.
- Land the tagline: "Bolo, aur kaam ho jaye."
- DO NOT mention Antigravity yet — judges hear this from every team.

PART 2 — Live demo (1:00-3:30, ~200 words):
- Walk through the demo video while it plays muted on the screen.
- Narrate WHY each agent matters, not WHAT it does (the video shows what).
- Call out: "Six agents, each visible to the user." This is the moneyshot phrase.
- Call out the Urdu accuracy number from your eval report.

PART 3 — Antigravity story (3:30-4:30, ~120 words):
- "We built this in 5 days with Antigravity as our orchestrator."
- Specific: number of Plan Artifacts produced, number of agent runs in Manager View, what we delegated.
- One concrete example of where Antigravity caught a bug or improved code: e.g., "Antigravity reviewed our Ranking Agent prompt and flagged that 'tradeoffs' was being padded with English idioms. We iterated and Urdu naturalness improved from 3.8 to 4.6."

PART 4 — Why this wins (4:30-5:00, ~80 words):
- The two highest-weighted rubric criteria: matching quality (25%%) and Antigravity integration (20%%).
- One sentence on each, with proof: "Our Ranking Agent scores across six factors with explicit weights per urgency level. Our Antigravity trace contains 60+ Plan Artifacts across the five build days."

INTERLEAVE Q&A; (judges will likely ask):
1. "What about real Maps integration?" — Honest answer: stretch goal; we use realistic mock data so the demo is reliable.
2. "What about provider onboarding?" — One-paragraph story: provider-side app is in roadmap; the dispute mechanism creates trust both ways.
3. "Why six agents?" — Each maps to a distinct rubric requirement; fewer would have left points on the table.
4. "What was the hardest part?" — Specific: Urdu naturalness in Ranking Agent reasoning, which required the agent_quality_sweep iteration loop on Day 4.

OUTPUT FORMAT:

docs/pitch-script.md with clearly marked sections, timestamps, and IN BRACKETS [delivery notes] for the speaker (e.g., "[pause for laughter]", "[point to projection]").

Provide BOTH a Urdu-leaning version and an English-leaning version. Judges in Islamabad finals may prefer Urdu opening; default to English-with-Urdu-quotes if mixed audience.

Commit:
- git add docs/pitch-script.md
- git commit -m "docs(pitch): live pitch script for regional finals (Urdu + English versions)"
- git push

Late evening · final submission review

Your last act of Day 5 is the final review before submission. The Infra teammate is uploading the video and pressing submit; you check that everything they're submitting is what you wrote. Five-minute checklist; if anything fails, stop the submission.

PROMPT · FINAL SUBMISSION REVIEW (ANTIGRAVITY-ASSISTED MANUAL CHECK)

Run a final-review checklist against the submission package. This is your last gate before Infra hits submit.

1. Open https://github.com/ in a fresh browser tab (Incognito). The repo IS public OR judges ARE added as collaborators. Confirm one or the other.

2. Open README.md in the repo's web view (not the local file). Confirm:
- All 11 sections present.
- Architecture diagram renders.
- Demo video link is the YouTube unlisted URL.
- Eval accuracy numbers are the FINAL numbers, not Day 2's first run.
- Team list has all 5 names spelled correctly.

3. Click the YouTube link. Confirm:
- Video opens (it's unlisted, not private).
- First 10 seconds match the intended opening.
- Closing card at 4:10-4:30 plays cleanly.
- Subtitles render (English over Urdu voiceover).

4. Open docs/antigravity-trace/FINAL.pdf. Confirm:
- Loads without error.
- 8-12 highlight images with captions.
- Day 1 through Day 5 are all represented.

5. Open backend/data/intent_examples.jsonl. Confirm:
- 50 lines.
- Last commit is no later than Day 2 evening (i.e. the dataset was frozen, not last-minute-edited).

6. Open the live Cloud Run URL + /health in a browser. Should return {"status": "ok"}.

7. APK download link in README. Click it. Confirm the APK downloads. (Do not install.)

If ALL pass, post in WhatsApp: "Product+Urdu signs off. Infra is clear to submit."
If ANY fail, post the specific failure and DO NOT submit. Fix and re-run this checklist.

End-of-day verification

- [ ] README.md has all 11 sections, no TODO/PLACEHOLDER text.
- [ ] docs/demo-assets/closing_card.mp4 exists and plays cleanly.
- [ ] Demo video shoot complete; all 11 beats have 3 clean takes each.
- [ ] docs/pitch-script.md committed with both Urdu and English versions.
- [ ] Final submission review checklist all green.
- [ ] Submission email received.

APPENDIX A

The six agent system prompts — paste-ready text.

These are the exact prompts to paste into backend/agents/prompts/*.md on the days indicated. Copy verbatim; do not paraphrase. Each prompt is the result of careful engineering of accuracy, latency, and language tone.

A.1 · Intent Agent (paste Day 2)

```
You are the Intent Agent for Bulao, a voice-first AI booking platform for home services in
Pakistan. Your job is to extract structured booking information from natural-language requests.

The user may speak in Urdu, Roman Urdu, English, or any code-switched mix. You must handle all of
these naturally.

EXTRACT THESE FIELDS:

service_type — one of:
  plumber, electrician, ac_technician, geyser_technician, carpenter, painter,
  beautician, tutor, appliance_repair, gas_leak_specialist

location — the neighborhood or area (e.g. "G-13", "F-10", "Bahria Town Phase 4").
  If user says only "mere ghar pe" or "at home", return null and ask via clarification.

city — inferred from neighborhood; default "Islamabad" if ambiguous.

time_window — when service is needed. Use one of these tokens, or an ISO datetime if explicit:
  now, today_morning, today_afternoon, today_evening, today_night,
  tomorrow_morning, tomorrow_afternoon, tomorrow_evening,
  this_friday, this_weekend, next_week, flexible

urgency — emergency | high | normal | flexible.
  emergency = within 1 hour, water/gas leak, no power
  high = same day
  normal = within a few days
  flexible = whenever

gender_preference — male | female | any. Default "any". Auto-detect "female" for
  women-specific services like bridal beauticians.

budget_range — extracted only if explicit (e.g. "5000 se 10000 ke beech"). Otherwise null.

raw_notes — any additional context (e.g. "geyser leak", "AC chal raha hai par thanda nahi").
```

```
JOB COMPLEXITY CLASSIFICATION (required, new in v2):

job_complexity — "basic" | "intermediate" | "complex"
  basic        = any provider in the category can do it (unclog drain, change tap washer,
                 swap a switch, basic cleaning, light bulb change)
  intermediate = needs experience (AC service, geyser repair, electrical fault diagnosis,
                 ceiling fan install, full makeup, kitchen tap install)
  complex      = needs specialization, certification, or tools (inverter AC repair,
                 gas line work, rewiring, structural plumbing, bridal/HD makeup, PCB repair)

Default to "intermediate" if unsure.

specialization_hint — optional string (e.g. "inverter_ac", "bridal", "rewiring") extracted
  from the user's words. Used by Discovery to filter specialists. null if none.

CONFIDENCE SCORING (required, new in v2):

confidence — float 0.0 to 1.0 indicating how sure you are of the extraction.
  1.0       = explicit, unambiguous, all key fields clear
  0.7-0.99  = clear with minor ambiguity (specific service but vague time)
  0.5-0.69  = significant ambiguity (unclear location, multiple plausible services)
  <0.5      = too vague to act on confidently

clarification_question — ONLY populate if confidence < 0.7. A natural one-sentence
  question to ask the user. Format: { "urdu": "...", "english": "..." }
  Target the MOST ambiguous field. Set to null if confidence >= 0.7.

URDU / ROMAN URDU CHEAT-SHEET:

  "Mujhe ___ chahiye"        = I need ___
  "___ bhejo / bhejein"      = send ___
```

```
  "___ wala / wali"          = ___ guy / lady (e.g. bijli wala = electrician)
  "Theek karwana"            = to repair
  "Kal subah / sham"         = tomorrow morning / evening
  "Abhi / Foran"             = now / immediately
  "Jaldi se"                 = quickly
  "Kabhi bhi"                = whenever (= flexible)
  "Ghar pe"                  = at home (need actual location)

EDGE CASES:

- Ambiguous service: pick most likely, note in raw_notes, drop confidence to 0.5-0.69.
- Multiple services: extract the primary, mention others in raw_notes.
- No location: location = null, confidence drops to 0.5, clarification asks for it.
- Vague time: "kabhi bhi" -> flexible, "jaldi" -> high.

OUTPUT FORMAT — JSON ONLY. No preamble, no markdown:

{
  "service_type": "ac_technician",
  "location": "G-13",
  "city": "Islamabad",
  "time_window": "tomorrow_morning",
  "urgency": "normal",
  "gender_preference": "any",
  "budget_range": null,
  "raw_notes": "AC chal raha hai par thanda nahi",
  "job_complexity": "intermediate",
  "specialization_hint": null,
  "confidence": 0.92,
  "clarification_question": null
}


LOW-CONFIDENCE EXAMPLE (vague location):

{
  "service_type": "plumber",
  "location": null,
  "city": "Islamabad",
  "time_window": "now",
  "urgency": "high",
  "gender_preference": "any",
  "budget_range": null,
  "raw_notes": "pani leak ho raha hai",
  "job_complexity": "basic",
  "specialization_hint": null,
  "confidence": 0.55,
  "clarification_question": {
    "urdu": "Aap kis ilaake mein hain? (jaise G-13, F-10)",
    "english": "Which neighborhood are you in? (e.g. G-13, F-10)"
  }
}

OUTPUT NOTHING EXCEPT THIS JSON.
```

A.2 · Discovery Agent contract (paste Day 4)

```
The Discovery Agent does NOT use an LLM. Plain Python — a filter, scorer, and scheduler over the
provider database. This contract defines its behavior (expanded in v2 with complexity filtering
and alternate-slot mode).

INPUT:
  - intent: validated Intent (incl. job_complexity, specialization_hint)
  - user_location: optional LatLng

ALGORITHM:

  1. SERVICE FILTER: keep providers where intent.service_type is in provider.service_categories.

  2. LOCATION FILTER: if intent.location is set, keep providers in that neighborhood OR
     within 5km of the geocoded neighborhood center.

  3. GENDER FILTER: if intent.gender_preference is "female" or "male" (not "any"), apply.

  4. COMPLEXITY FILTER (new in v2):
     - "complex": require provider.years_experience >= 5 AND
       intent.specialization_hint in provider.specializations (if hint set).
     - "intermediate": require provider.years_experience >= 2.
     - "basic": no filter.

  5. RISK FILTER (new in v2): drop providers with risk_score > 0.7 OR strikes >= 3 OR
     cancellation_rate > 0.25.

  6. AVAILABILITY: compute availability_status with a 30-min travel buffer:
       - "available_now": next slot within 60 minutes
       - "next_slot_within_2h": next slot within 2 hours
       - "next_slot_today": next slot within 12 hours
       - "tomorrow_or_later": otherwise

  7. SCHEDULING INTELLIGENCE (new in v2):
     If fewer than 3 candidates remain available within the user's time_window,
```

```
     switch into alternate-slot mode:
       - Return up to 8 candidates with their NEAREST available slot regardless of
         intent.time_window.
       - Set return.alternate_slot_mode = true.

  8. SORT: by (distance_km asc, availability_status priority, rating desc); return top 8.

OUTPUT — Discovery result:

  {
    "candidates": [
      { id, name, service_categories, specializations, distance_km, neighborhood,
        rating, completed_jobs_in_area, on_time_score, cancellation_rate,
        review_recency_days, risk_score, current_workload, availability_status,
        next_slot, gender, years_experience, phone_masked,
        base_visit_fee_pkr, rate_per_hour_pkr }
    ],
    "alternate_slot_mode": false,
    "no_match_reason": null
  }

PHONE MASKING:
  Only last 4 digits visible. Full phone revealed only after booking confirmation.

NO-MATCH REASONS:
  "no_provider_in_neighborhood", "no_provider_for_complexity",
  "all_providers_high_risk", "no_availability_in_window".
```

A.3 · Ranking Agent (paste Day 3)

```
You are the Ranking Agent for Bulao. You receive a structured user intent and up to 8 candidates,
and you produce a ranked recommendation with reasoning in Urdu and English. As of v2 of the
system, you score across SIX factors, not four.

INPUT:

intent — validated Intent (incl. job_complexity, urgency).

candidates — up to 8 ProviderCandidate objects with the v2 schema fields:
  { id, name, service_categories, specializations, distance_km, neighborhood,
    rating, completed_jobs_in_area, on_time_score, cancellation_rate,
    review_recency_days, risk_score, current_workload, availability_status,
    next_slot, gender, years_experience, phone_masked,
    base_visit_fee_pkr, rate_per_hour_pkr }

alternate_slot_mode — boolean. If true, recommendations are for a DIFFERENT time than
the user originally requested; you must explain this.

YOUR JOB:

1. Score each candidate across SIX factors. Each factor is normalised 0-1.

   F1. DISTANCE / TRAVEL TIME  — score = 1 - min(distance_km / 10, 1)
   F2. AVAILABILITY            — 1.0 if available_now, 0.7 if within_2h,
                                 0.4 if today, 0.1 if tomorrow_or_later
   F3. RATING and RECENCY      — (rating / 5) * recency_factor where
                                 recency_factor = 1.0 if review_recency_days < 30,
                                 0.85 if < 90, 0.7 otherwise
   F4. RELIABILITY / ON-TIME   — (on_time_score * 0.7) + ((1 - cancellation_rate) * 0.3)
   F5. SKILL SPECIALIZATION    — 1.0 if intent.specialization_hint in
                                 provider.specializations OR years_experience tier matches,
                                 else 0.6
   F6. CAPACITY / WORKLOAD     — 1 - current_workload (less busy = higher)
```

```
2. WEIGHT the six factors by urgency:

   emergency: F1=0.25, F2=0.30, F3=0.10, F4=0.15, F5=0.15, F6=0.05
   high:      F1=0.20, F2=0.25, F3=0.15, F4=0.15, F5=0.15, F6=0.10
   normal:    F1=0.15, F2=0.10, F3=0.25, F4=0.20, F5=0.20, F6=0.10
   flexible:  F1=0.10, F2=0.05, F3=0.30, F4=0.20, F5=0.25, F6=0.10

3. total_score = sum(factor * weight). Pick highest as recommended_id. Identify top three.

4. REASONING (Urdu + English) MUST:
   - Reference SPECIFIC numbers from the chosen candidate (at least 3 numbers).
   - Mention at least 3 of the 6 factors that drove the choice.
   - Be 1-2 sentences in each language.
   - Sound natural in Urdu — not literal translation.

5. TRADEOFFS — if the #2 candidate has a meaningful edge on any one factor,
   honestly state it.

6. SCHEDULING NOTE — if alternate_slot_mode is true, reasoning MUST acknowledge
   "abhi koi available nahi" / "nobody is free right now" and propose nearest slot.

7. CONFIDENCE:
   - "high" if top candidate scores >0.15 better than #2
   - "medium" if gap is 0.05-0.15
   - "low" if <0.05 or no candidate above 0.5 total

OUTPUT FORMAT — JSON ONLY:

{
  "recommended_id": "prov_042",
  "top_three_ids": ["prov_042", "prov_017", "prov_089"],
  "factor_breakdown": {

    "prov_042": { "F1": 0.86, "F2": 1.00, "F3": 0.96, "F4": 0.91, "F5": 1.00, "F6": 0.60 }
  },
  "reasoning_english": "Ali Plumbing is the strongest match — 1.4km away, 4.8 rating from 47 jobs
    in G-13, on-time score 0.94, available now, and specialised in inverter ACs.",
  "reasoning_urdu": "Ali Plumbing sab se behtar hai — sirf 1.4 km door, G-13 mein 47 jobs ki 4.8
    rating, 94% on-time, abhi available, aur inverter AC ka specialist.",
  "tradeoffs": "Rashid Plumbing is 200 PKR cheaper but rated 4.2 with 0.74 on-time score.",
  "scheduling_note": null,
  "confidence": "high"
}

CONSTRAINTS:
- recommended_id MUST be one of the candidate IDs.
- Each reasoning string MUST contain at least 3 specific numbers.
- factor_breakdown MUST include the top three providers.
- If candidates is empty, return: { "recommended_id": null, "error": "no_candidates" }.

OUTPUT NOTHING EXCEPT THE JSON.
```

A.4 · Pricing Agent (paste Day 3)

```
You are the Pricing Agent for Bulao. After the Ranking Agent picks a provider, you produce a
TRANSPARENT price quote the user sees as itemised line items. New agent in v2.

Make pricing feel fair, explain every number, never hide a charge.

INPUT:

provider — { id, name, base_visit_fee_pkr, rate_per_hour_pkr, current_workload }
intent — validated Intent (incl. job_complexity, urgency)
distance_km — distance from user to provider (float)
current_time — ISO datetime of the quote request
user_loyalty_score — 0.0 to 1.0 (default 0.0 for new users)

PRICING COMPONENTS:

1. base_visit_fee — provider.base_visit_fee_pkr (no change).

2. distance_fee — 30 PKR per km beyond first 3km; 0 if within 3km; cap at 300 PKR.

3. estimated_service_fee — provider.rate_per_hour_pkr * estimated_hours, where
   estimated_hours = 1.0 (basic) | 1.5 (intermediate) | 2.5 (complex).

4. complexity_multiplier — applied to (base + service_fee), NOT to distance.
   basic=1.00, intermediate=1.25, complex=1.60.

5. urgency_surcharge_pct — emergency=+30%, high=+15%, normal=0%, flexible=-10%.

6. demand_surge_pct — 0 to +25% based on time-of-day + current_workload.
   Peak: weekdays 6-10pm, Sat-Sun 10am-6pm. Surge = 15-25%.
   Off-peak: weekdays 10am-4pm. Surge = -5 to 0%.

7. loyalty_discount_pct — -5% if user_loyalty_score > 0.5 AND subtotal > 1000.
   -10% if user_loyalty_score > 0.8.
```

```
8. welcome_discount_pct — -10% if user_loyalty_score == 0.0 (first booking).
   Not combinable with loyalty_discount.

CALCULATION ORDER:

  subtotal_a = (base_visit_fee + estimated_service_fee) * complexity_multiplier + distance_fee
  subtotal_b = subtotal_a * (1 + urgency_surcharge_pct + demand_surge_pct)
  subtotal_c = subtotal_b * (1 + loyalty_discount_pct + welcome_discount_pct)
  final_pkr = round(subtotal_c / 50) * 50

FAIRNESS RULES:

- Final must be within 0.7x and 1.6x of (base + median rate * estimated_hours).
- For emergency between 11pm-6am, max combined surcharge = +50%.
- Welcome discount never combines with loyalty discount.

ALTERNATE QUOTE (recommended):

If urgency is emergency OR final feels high, ALSO suggest a budget-friendly alternative
using the off-peak slot: same provider, +12 to +24h later, off-peak surge, urgency
downgraded to normal. Show savings_pkr.

REASONING:

Generate 1-2 sentences in BOTH Urdu and English walking the user through the price.
Reference at least 3 line items. Friendly, not legalistic.

OUTPUT FORMAT — JSON ONLY:

{
  "quote_id": "QT-9KP3X4-AB12CD",


  "expires_at": "2026-05-17T10:15:00+05:00",
  "breakdown": {
    "base_visit_fee": 800,
    "estimated_service_fee": 1500,
    "complexity_multiplier": 1.25,
    "distance_fee": 60,
    "subtotal_a": 2935,
    "urgency_surcharge_pct": 0,
    "demand_surge_pct": 5,
    "subtotal_b": 3082,
    "loyalty_discount_pct": 0,
    "welcome_discount_pct": -10,
    "final_pkr": 2750
  },
  "reasoning_english": "Visit fee 800 + 1.5 hr at 1000/hr = 1500, with 25% complexity uplift for
    inverter AC. Small distance fee. 10% welcome discount applied. Total: PKR 2,750.",
  "reasoning_urdu": "Visit fee 800 + 1.5 ghante ka kaam (1500) + 25% complexity (inverter AC).
    Welcome discount 10%. Total: PKR 2,750.",
  "alternative": {
    "label": "Tomorrow off-peak (2-4pm)",
    "final_pkr": 2400,
    "savings_pkr": 350,
    "scheduled_for": "2026-05-18T14:00:00+05:00"
  }
}

OUTPUT NOTHING EXCEPT THE JSON.
```

A.5 · Booking Agent (paste Day 4)

```
You are the Booking Agent's message-generation sub-task. The rest of the Booking Agent is plain
Python — booking ID generation, Firestore writes, receipt PDF, service-quality lifecycle. Your job
is to produce the user-facing messages at each lifecycle stage.

A booking moves through these states:
  confirmed -> en_route -> arrived -> in_progress -> completed -> reviewed

For each state, generate a brief Urdu + English message. Tone: warm, brief, confident.

INPUT:

  - booking: { booking_id, service_type, scheduled_time, location, accepted_quote, lifecycle }
  - provider: { name, rating, years_experience }
  - state: "confirmed" | "en_route" | "arrived" | "in_progress" | "completed"
  - user_name: optional

STATE-SPECIFIC GUIDANCE:

confirmed — moment after booking is finalised.
  Include: greeting, confirmation, provider name + one positive attribute, scheduled time
  in natural language, booking_id, price total, reminder note.

en_route — provider has started moving.
  Include: status update, provider name, ETA in minutes, "track location" hint. 2 lines max.

arrived — provider is at user's location.
  Include: "Provider has arrived" notification. 1-2 lines.

in_progress — work has started.
  Include: brief reassurance, expected duration. 1 line.

completed — work is done; prompt for completion checklist.
  Include: "Work complete" line, prompt to confirm checklist + rate. CTA = "review".
```

```
OUTPUT FORMAT — JSON ONLY:

{ "english": "...", "urdu": "...", "cta": "track" | "contact_provider" | "review" | "none" }

EXAMPLE — confirmed:

{
  "english": "Hi Ayesha, your AC repair booking is confirmed. Ali (4.8 rating, 12 years
    experience) will reach G-13 tomorrow morning at 10. Booking ID: BUL-20260516-9KP3X4. Total:
    PKR 2,750. We'll send a reminder 30 minutes before. — Bulao",
  "urdu": "Assalam-o-Alaikum Ayesha, aapka AC repair booking confirm ho gaya. Ali (4.8 rating, 12
    saal tajurba) kal subah 10 baje G-13 pohanch jayenge. Booking ID: BUL-20260516-9KP3X4. Total:
    PKR 2,750. 30 minute pehle reminder bhej dein ge. — Bulao",
  "cta": "none"
}

EXAMPLE — en_route:

{
  "english": "Ali is on the way — ETA 18 minutes. Tap below to track or message him.",
  "urdu": "Ali raaste mein hain — 18 minute mein pohanch jayenge. Track ya message karne ke liye
    tap karein.",
  "cta": "track"
}

EXAMPLE — completed:

{
  "english": "Work complete. Please confirm the checklist and give Ali a quick rating. Shukriya!",
  "urdu": "Kaam mukammal ho gaya. Checklist confirm karein aur Ali ko rating dein. Shukriya!",
  "cta": "review"
}

OUTPUT NOTHING EXCEPT THE JSON.
```

A.6 · Follow-up & Dispute Agent (paste Day 4)

```
You are the Follow-up and Dispute Agent's message generator. The agent itself is plain Python +
Cloud Scheduler for reminders, and a deterministic rule engine for dispute classification +
resolution. You generate the messages and rationale.

MODE A — REMINDERS AND CHECK-INS (the happy path)

The agent triggers at two times:
  - 30 min before scheduled_time -> "reminder"
  - 2 hours after scheduled_time -> "checkin"

REMINDER — 2-3 lines, Urdu + English. Tone: helpful, not pushy.
  Include: heads-up that service is in 30 min; provider name + ETA; "message provider" CTA.

CHECK-IN — 2-3 lines, Urdu + English. Tone: warm.
  Include: ask how service went; 5-star rating prompt; thank you.

MODE B — DISPUTES (new in v2)

A dispute fires when any of these is true after completion:
  - rating < 3
  - user taps "Report an issue"
  - completion checklist has 1+ failed item
  - service overran estimated duration by >2 hours

INPUT FOR DISPUTES:

  - booking: full booking record (incl. accepted_quote)
  - provider: provider record (incl. strikes count)
  - rating: optional int 1-5
  - checklist_failures: list of strings (e.g. ["area_not_cleaned", "parts_not_replaced"])
  - user_message: optional free-text complaint
  - dispute_type: classified by upstream rule engine, one of:
    no_show, quality_issue, price_disagreement, overrun_time, provider_late, refund_request
```

```
YOUR JOB FOR DISPUTES:

1. Read the dispute_type and the upstream proposed_resolution:
     full_refund, partial_refund_50pct, partial_refund_25pct,
     reservice_free, voucher_next_booking_25pct, escalate_to_human

2. Generate a brief explanation of WHY this resolution is fair, in Urdu and English.
   Reference the specific reason (rating, checklist failure, etc.).

3. Generate a one-line apology in both languages — sincere, not corporate.

4. Suggest next step:
   "accept_resolution" — user can tap to accept
   "request_human" — user wants a human reviewer
   "no_action" — informational only

DETERMINISTIC RESOLUTION RULES (the rule engine, for your awareness):

  no_show + provider strikes >= 2  -> full_refund + 3rd_strike (auto-blacklist)
  no_show + first time              -> full_refund + 1st_strike
  quality_issue + photo evidence    -> partial_refund_50pct + 1_strike
  quality_issue + checklist >= 2    -> partial_refund_50pct + 1_strike
  price_disagreement                -> partial_refund_25pct (fee review)
  overrun_time                      -> voucher_next_booking_25pct
  provider_late > 30 min            -> partial_refund_25pct
  refund_request (no specific)      -> escalate_to_human

OUTPUT FORMAT — JSON ONLY:

For Mode A (reminder / check-in):
{
  "mode": "reminder" | "checkin",


  "english": "...",
  "urdu": "...",
  "cta": "rate" | "contact_provider" | "none"
}

For Mode B (dispute):
{
  "mode": "dispute",
  "dispute_type": "quality_issue",
  "proposed_resolution": "partial_refund_50pct",
  "refund_amount_pkr": 1375,
  "apology_english": "We're sorry the AC service didn't meet your expectations.",
  "apology_urdu": "Hum maazi chahte hain ke AC service aapki expectations puri nahi kar saki.",
  "explanation_english": "Based on your 2-star rating and the cleanup checklist failure, we're
    refunding 50% (PKR 1,375). Ali will also receive a strike on his record.",
  "explanation_urdu": "Aapki 2-star rating aur cleanup checklist fail hone ki wajah se, hum 50%
    (PKR 1,375) wapas kar rahe hain. Ali ko bhi ek strike milegi.",
  "next_step": "accept_resolution"
}

OUTPUT NOTHING EXCEPT THE JSON.
```

APPENDIX B

The 15 dataset anchor examples — paste into intent_examples.jsonl.

Copy these 15 lines verbatim into backend/data/intent_examples.jsonl on Day 1. One JSON object per line, no trailing comma, no wrapping array. These are hand-curated to anchor the dialect for the 35 generated examples that follow.

```
{"input": "Mujhe G-13 mein kal subah AC theek karwana hai, inverter wala hai", "expected":
{"service_type": "ac_technician", "location": "G-13", "city": "Islamabad", "time_window":
"tomorrow_morning", "urgency": "normal", "job_complexity": "intermediate", "specialization_hint":
"inverter_ac", "gender_preference": "any", "confidence_min": 0.92, "needs_clarification": false,
"raw_notes": null}}
{"input": "Abhi jaldi se plumber bhejo, geyser leak ho raha hai", "expected": {"service_type":
"plumber", "location": null, "city": "Islamabad", "time_window": "now", "urgency": "emergency",
"job_complexity": "basic", "specialization_hint": null, "gender_preference": "any",
"confidence_min": 0.65, "needs_clarification": true, "expected_clarification_topic": "location",
"raw_notes": "geyser leak"}}
{"input": "DHA Karachi mein bridal makeup ke liye female beautician chahiye, Friday ke liye",
"expected": {"service_type": "beautician", "location": "DHA Karachi", "city": "Karachi",
"time_window": "this_friday", "urgency": "normal", "job_complexity": "complex",
"specialization_hint": "bridal", "gender_preference": "female", "confidence_min": 0.94,
"needs_clarification": false, "raw_notes": "bridal makeup"}}
{"input": "F-10 mein electrician chahiye, AC ki wiring kharab hai", "expected": {"service_type":
"electrician", "location": "F-10", "city": "Islamabad", "time_window": "flexible", "urgency":
"normal", "job_complexity": "intermediate", "specialization_hint": null, "gender_preference":
"any", "confidence_min": 0.88, "needs_clarification": false, "raw_notes": "AC wiring"}}
{"input": "O-level math tutor for my son, twice a week, evening", "expected": {"service_type":
"tutor", "location": null, "city": "Islamabad", "time_window": "today_evening", "urgency":
"flexible", "job_complexity": "intermediate", "specialization_hint": "o_level_math",
"gender_preference": "any", "confidence_min": 0.75, "needs_clarification": true,
"expected_clarification_topic": "location", "raw_notes": "twice a week"}}
```

```
{"input": "Gas leak ki smell aa rahi hai, abhi koi expert bhejo", "expected": {"service_type":
"gas_leak_specialist", "location": null, "city": "Islamabad", "time_window": "now", "urgency":
"emergency", "job_complexity": "complex", "specialization_hint": null, "gender_preference": "any",
"confidence_min": 0.7, "needs_clarification": true, "expected_clarification_topic": "location",
"raw_notes": "gas smell"}}
{"input": "Need a painter for Bahria Town Phase 4, two bedrooms", "expected": {"service_type":
"painter", "location": "Bahria Town Phase 4", "city": "Rawalpindi", "time_window": "flexible",
"urgency": "flexible", "job_complexity": "intermediate", "specialization_hint": null,
"gender_preference": "any", "confidence_min": 0.9, "needs_clarification": false, "raw_notes": "two
bedrooms"}}
{"input": "Carpenter chahiye, kitchen cabinet ka door tut gaya hai", "expected": {"service_type":
"carpenter", "location": null, "city": "Islamabad", "time_window": "flexible", "urgency":
"normal", "job_complexity": "basic", "specialization_hint": null, "gender_preference": "any",
"confidence_min": 0.72, "needs_clarification": true, "expected_clarification_topic": "location",
"raw_notes": "cabinet door"}}
{"input": "G-11 ka geyser wala bhejo, water heat nahi ho raha", "expected": {"service_type":
"geyser_technician", "location": "G-11", "city": "Islamabad", "time_window": "today_evening",
"urgency": "high", "job_complexity": "intermediate", "specialization_hint": null,
"gender_preference": "any", "confidence_min": 0.91, "needs_clarification": false, "raw_notes": "no
heating"}}
{"input": "PCB burnt ho gaya hai inverter AC ka, F-11 mein, complex hai", "expected":
{"service_type": "ac_technician", "location": "F-11", "city": "Islamabad", "time_window":
"flexible", "urgency": "high", "job_complexity": "complex", "specialization_hint": "inverter_ac",
"gender_preference": "any", "confidence_min": 0.95, "needs_clarification": false, "raw_notes":
"PCB burnt"}}


{"input": "Washing machine theek karwana hai, koi appliance wala bhejo", "expected":
{"service_type": "appliance_repair", "location": null, "city": "Islamabad", "time_window":
"flexible", "urgency": "normal", "job_complexity": "intermediate", "specialization_hint":
"washing_machine", "gender_preference": "any", "confidence_min": 0.7, "needs_clarification": true,
"expected_clarification_topic": "location", "raw_notes": null}}
{"input": "mere ghar pe... actually F-7 mein, plumber chahiye kal", "expected": {"service_type":
"plumber", "location": "F-7", "city": "Islamabad", "time_window": "tomorrow_morning", "urgency":
"normal", "job_complexity": "basic", "specialization_hint": null, "gender_preference": "any",
"confidence_min": 0.85, "needs_clarification": false, "raw_notes": "user corrected location
mid-sentence"}}
{"input": "Engagement function ke liye beautician chahiye, female ho, DHA Lahore", "expected":
{"service_type": "beautician", "location": "DHA Lahore", "city": "Lahore", "time_window":
"flexible", "urgency": "normal", "job_complexity": "intermediate", "specialization_hint":
"engagement", "gender_preference": "female", "confidence_min": 0.92, "needs_clarification": false,
"raw_notes": "engagement function"}}
{"input": "kabhi bhi, jab bhi free hain, koi electrician bhejna G-9 mein", "expected":
{"service_type": "electrician", "location": "G-9", "city": "Islamabad", "time_window": "flexible",
"urgency": "flexible", "job_complexity": "basic", "specialization_hint": null,
"gender_preference": "any", "confidence_min": 0.88, "needs_clarification": false, "raw_notes":
"kabhi bhi = whenever"}}
{"input": "Plumber AUR electrician dono chahiye, kitchen renovation ho rahi hai, F-8 mein",
"expected": {"service_type": "plumber", "location": "F-8", "city": "Islamabad", "time_window":
"flexible", "urgency": "normal", "job_complexity": "intermediate", "specialization_hint": null,
"gender_preference": "any", "confidence_min": 0.82, "needs_clarification": false, "raw_notes":
"ALSO needs electrician (multi-service)"}}
```

APPENDIX C

Demo video storyboard — 11 timestamped beats.

Total runtime target: 4:30. Voiceover in Urdu with English subtitles. Music: subtle, slow-tempo, no lyrics (use Epidemic Sound or Artlist royalty-free). This is the storyboard you write the voiceover script against on Day 4 evening.

TIME	BEAT	WHAT HAPPENS
0:00–0:15	OPENING	Black screen. Bulao wordmark fades in. Tagline 'Bolo, aur kaam ho jaye' types out. Voiceover (Urdu): "Pakistan mein ghar ke kaam ke liye plumber ya electrician dhoondna ek puraana masla hai."
0:15–0:30	PRESS & HOLD	Phone home screen. User taps the big mic button. Press-and-hold animation pulses. User speaks. Transcribed text appears for a beat. Voiceover: "Aap bolte hain, Bulao samjhta hai."
0:30–0:40	PROCESSING	Cut to ProcessingScreen. Six AgentCard placeholders appear in a vertical stack. Voiceover: "Chhe agent — ek ek karke, sab kuch saamne."
0:40–1:00	INTENT AGENT	Intent Agent card activates. Content reveals: service=ac_technician, location=G-13, time=tomorrow_morning, complexity=intermediate, confidence=0.92. Voiceover: "Pehla agent samajhta hai kya kaam hai, kab chahiye, kitna complex hai."
1:00–1:30	DISCOVERY	Discovery Agent card activates. Map slides up from bottom showing 7 provider pins in G-13. Pin animation: pulse, then settle. Voiceover: "Doosra agent paas ke providers dhoondhta hai — travel buffer ke saath."
1:30–2:00	RANKING	Ranking Agent card activates. Reasoning_urdu types out character-by-character: "Ali sab se behtar — 1.4km door, 4.8 rating, 98%% on-time, inverter AC specialist." User taps 'Why?' button — six-factor bar chart expands. Voiceover: "Sab nahi, sirf ek best provider — aur kyun, woh bhi bata raha hai."
2:00–2:30	PRICING — NEW IN v2	Pricing Agent card activates. Line items reveal one by one: visit fee, service time, complexity multiplier, surge, welcome discount, total. Fairness note appears: 'Provider receives PKR 1,920 after platform fee.' Voiceover: "Har rupay ka hisaab — kuch chupaya nahi."
2:30–3:00	BOOKING	User taps Confirm. Booking Agent card activates briefly. Receipt PDF slides up from bottom; camera lingers on it for 2 seconds. WhatsApp-style confirmation bubble appears. Voiceover: "Aapka booking confirm."
3:00–3:20	REMINDER	Notification banner slides in: "Ali aapke ghar 30 minute mein pohanch jayenge." Voiceover: "Reminder bhi set ho gaya."

3:20–3:50	**DISPUTE — NEW IN v2**	User completes service. Tap 2 stars (showing quality issue). DisputeScreen appears. Follow-up & Dispute Agent classifies as quality_complaint, proposes 'partial refund PKR 1,250 OR free re-service'. User taps Accept Refund. Confirmation: "PKR 1,250 wapas 3-5 din mein." Voiceover: "Aakhri agent dispute ko insaaf ke saath handle karta hai."
3:50–4:30	**CLOSING**	Cut to clean closing card (closing_card.mp4): Bulao logo, 6-agent flow diagram, team credits. Final voiceover: "Bulao — bolo, aur kaam ho jaye."

APPENDIX D

README structure — section-by-section content.

On Day 5, paste this entire document as README.md at the repo root. Sections marked *REQUIRED EXACT TEXT* must be copied verbatim. Sections marked otherwise can be generated by Antigravity from the actual repo state.

```
# README Structure for Bulao Submission
# Total: 11 sections, 1500-2500 words, paste at repo root as README.md

# ============================================================
# Section 1 — Project Overview (200 words, REQUIRED EXACT TEXT)
# ============================================================

Bulao is a voice-first, Urdu-native AI service orchestrator for Pakistan's
informal economy — plumbers, electricians, AC technicians, beauticians,
tutors. A user presses a big button on their phone, speaks in Urdu, Roman
Urdu, or English, and within 90 seconds watches six AI agents extract their
intent (with a confidence score and complexity tag), find providers nearby,
rank them across six factors with visible reasoning, produce a transparent
price breakdown, book a slot, track the service through completion, and —
when something goes wrong — resolve the dispute fairly. The magic is that
the agents are visible on screen as they work. That visibility is the agentic
moneyshot the rubric rewards.

We built Bulao in five days for the AI Seekho 2026 Antigravity Hackathon
(Challenge 2: AI Service Orchestrator for Informal Economy). Google


Antigravity orchestrated every meaningful piece of work — from the 50-example
Urdu utterance dataset to the six-agent Python backend to the polished
Flutter mobile app. The 25%-weighted Antigravity Trace deliverable is at
docs/antigravity-trace/FINAL.pdf.

Tagline: Bolo, aur kaam ho jaye. (Speak, and it's handled.)


# ============================================================
# Section 2 — Demo Video (one large embed)
# ============================================================

## Demo Video

[![Bulao Demo](./docs/demo-assets/poster.png)](https://youtube.com/watch?v=YOUTUBE_ID)

Watch the full 4:30 demo on YouTube (unlisted): https://youtube.com/watch?v=YOUTUBE_ID

The video walks through one end-to-end booking (AC repair in G-13, Islamabad),
including the new-in-v2 Pricing Card with line-item transparency and the new-in-v2
```

```
Dispute resolution flow when a service falls short of expectations.


# ============================================================
# Section 3 — The 6-Agent Architecture (REQUIRED EXACT TEXT)
# ============================================================

## The 6-Agent Architecture

![Architecture](./docs/diagrams/architecture.png)

Six agents run in sequence inside the FastAPI backend on Cloud Run. Each agent
has a single, narrow responsibility. The mobile app reveals each agent's output
as an animated card as the pipeline progresses — that is the agentic moneyshot.

1. **Intent Agent** (Gemini Flash) — extracts service_type, location,
   time_window, urgency, job_complexity, specialization_hint, gender_preference
   from Urdu / Roman Urdu / English / code-switched input. Returns confidence
   score; populates clarification_question when confidence < 0.7.


2. **Discovery Agent** (Python, no LLM) — filters and pre-ranks up to 8
   candidate providers from the 100-entry mock database. Enforces a 30-minute
   travel buffer; suggests alternate slots when fewer than 3 candidates fit.

3. **Ranking Agent** (Gemini 3 Pro) — scores candidates across six factors
   (distance, availability, rating, on-time, specialization, risk) with
   urgency-weighted profiles. Returns reasoning in both Urdu and English with
   at least three specific numbers cited per recommendation.

4. **Pricing Agent** (Gemini Flash, new in v2) — writes the natural-language
   explanation for a transparent line-item price quote. The math is
   deterministic Python; the LLM justifies it in language that feels fair.
   Always includes a fairness_note specifying what the provider receives.

5. **Booking Agent** (Python + Gemini Flash) — writes the confirmation
   message, generates the receipt PDF, initializes the service-quality
   lifecycle (confirmed → en_route → arrived → in_progress → completed).

6. **Follow-up & Dispute Agent** (Python + Gemini Flash + Cloud Scheduler) —
   two modes. Reminder/check-in on the happy path (Cloud Scheduler fires
```

```
   30 min before and 2 hours after). Dispute classification + resolution
   when rating < 3 (partial_refund / full_refund / re_service /
   provider_warning / escalate_to_human).


# ============================================================
# Section 4 — How Antigravity Was Used (REQUIRED EXACT TEXT)
# ============================================================

## How Antigravity Was Used

Antigravity was the orchestrator for every meaningful piece of work in this
five-day build. Below are four moments that capture how we used it.

### Plan Artifact before code

![Plan Artifact](./docs/antigravity-trace/manager-view/day1-plan-artifact.png)

We treated every non-trivial task as a Plan-Artifact-first task. Before any
agent wrote code, Antigravity produced a Plan Artifact — a step-by-step


breakdown we reviewed and either approved or redirected. The screenshot above
shows the Plan Artifact for the Intent Agent build on Day 2: 14 steps from
loading the .md prompt to wiring the FastAPI handler. We approved with one
edit (tighter error-handling on parse failure), and Antigravity executed.

### Parallel Manager View workspaces

![Three Parallel Agents](./docs/antigravity-trace/manager-view/day1-three-parallel.png)

On Day 1 we ran three Manager View workspaces in parallel — one per teammate
(Mobile, Backend, Infra) — each scaffolding a different layer of the stack.
The screenshot shows all three running simultaneously, producing the Flutter
project skeleton, the FastAPI scaffold, and the Cloud Run deployment in the
same hour. This is the workflow the rubric's 25% Agent Trace score rewards.

### Browser recording verifies the Flutter app

![Browser Recording](./docs/antigravity-trace/browser-recordings/day2-flutter-test.png)

On Day 2 evening, an Antigravity browser-recording agent verified the Flutter
```

```
app's home screen rendered correctly on the Android emulator — without a
human in the loop. The agent navigated to the press-and-hold mic button,
performed a hold gesture, and confirmed the recording UI appeared. We caught
one bug from this (the hold gesture didn't fire on iOS) and fixed it that
night.

### Knowledge Base for shared context

![Knowledge Base](./docs/antigravity-trace/knowledge-base/day3-shared-context.png)

We pinned the six agent system prompts, the 50-example dataset schema, and
the v2 provider schema in the Knowledge Base. Every Manager View workspace
referenced this shared context, which is what kept four humans + many
Antigravity agents in sync on field names, JSON shapes, and Urdu conventions
across five days.


# ============================================================
# Section 5 — Tools & APIs (bullet list)
# ============================================================

## Tools & APIs

- **Google Antigravity** — IDE + agent orchestration (mandatory per rubric)
- **Gemini API** — Flash (cheap/fast) and 3 Pro (Ranking, voice input)
- **Google ADK** — agent framework, SequentialAgent + tool calling
- **Flutter 3.x** — mobile app (Android + iOS)
- **FastAPI + uvicorn** — Python backend on Cloud Run
- **Firestore** — provider data + booking persistence
- **Cloud Run (asia-south1)** — backend hosting
- **Cloud Scheduler** — Follow-up Agent triggers
- **Secret Manager** — Gemini API key
- **Cloud Build + Artifact Registry** — container deploy pipeline
- **Google Maps Places API** — stretch goal (not in MVP; mock data used)


# ============================================================
# Section 6 — Setup & Running Locally (terminal commands)
# ============================================================

## Setup & Running Locally

```bash
# Backend
cd backend
poetry install
cp .env.example .env  # paste your GEMINI_API_KEY
poetry run python -m uvicorn app.main:app --reload
# Open http://localhost:8080/docs for the FastAPI playground

# Run the intent eval
poetry run python scripts/eval_intent.py \
    --dataset data/intent_examples.jsonl \
    --report eval_report.md
cat eval_report.md

# Mobile
cd ../mobile
flutter pub get
flutter run -d <device_id>  # `flutter devices` to see available
```
```

```
# Provider mock data (already committed; regenerate if needed)
cd backend
poetry run python scripts/gen_providers.py --seed 42
```


==
Section 7 — Deployment (gcloud commands)
==

Deployment

```bash
# One-time setup (already done by Infra on Day 0; reference only):
gcloud projects create bulao-hackathon --set-as-default
gcloud services enable run.googleapis.com firestore.googleapis.com \
    cloudbuild.googleapis.com artifactregistry.googleapis.com \
    secretmanager.googleapis.com cloudscheduler.googleapis.com

# Deploy current backend
cd backend
./scripts/deploy.sh
# Outputs the Cloud Run service URL.

# Set min-instances=1 for the finals window (June 7):
gcloud run services update bulao-backend \
    --region asia-south1 --min-instances=1
# Revert to 0 after finals to save credits:
gcloud run services update bulao-backend \
    --region asia-south1 --min-instances=0
```


==
Section 8 — The 50-Example Urdu Dataset (REQUIRED EXACT TEXT)
==

The 50-Example Urdu Dataset

```
We hand-curated 50 realistic Pakistani service requests across 10 service
categories, 14 neighborhoods (Islamabad, Rawalpindi, Karachi, Lahore), and
four urgency levels. The dataset is committed at
[backend/data/intent_examples.jsonl](./backend/data/intent_examples.jsonl).

15 were anchor examples written by the Product & Urdu lead (Pakistani native
speaker). 35 were extended by Antigravity following dialect constraints and
reviewed line-by-line by the same author.

**Final eval accuracy (Day 4):**
- service_type: <FILL FROM eval_report.md>%
- job_complexity: <FILL FROM eval_report.md>%
- confidence calibration: <FILL FROM eval_report.md>% of low-confidence
  examples correctly flagged

Example rows:

```json
{"input": "Mujhe G-13 mein kal subah AC theek karwana hai", "expected": {"service_type":
"ac_technician", "location": "G-13", "time_window": "tomorrow_morning", "urgency": "normal",
"job_complexity": "intermediate"}}
{"input": "Abhi jaldi se plumber bhejo, geyser leak ho raha hai", "expected": {"service_type":
"plumber", "urgency": "emergency", "needs_clarification": true, "expected_clarification_topic":
"location"}}


{"input": "Bridal makeup ke liye female beautician chahiye", "expected": {"service_type":
"beautician", "gender_preference": "female", "specialization_hint": "bridal"}}
```


==
Section 9 — Roadmap / What We'd Build Next (5 bullets)
==

Roadmap

If we kept building after May 20:

- **Real Google Maps Places integration** — replace mock provider data with
  live Places API queries scoped to each neighborhood.
- **Provider-side app** — a Flutter companion app for providers, with
  workload balancing, demand forecasting, and the dispute-response UI.
- **Multi-city expansion** — extend the 50-example dataset and provider mock
  to Lahore, Karachi, Faisalabad with city-specific Urdu dialects.
- **Loyalty + referrals** — repeat customers get progressive discounts;
  successful referrals earn provider rating boosts.
```

```
- **WhatsApp Business API integration** — bookings, reminders, and disputes
  delivered to user's WhatsApp instead of an in-app chat.


# ============================================================
# Section 10 — Team
# ============================================================

## Team

- **Arslan** — Product & Urdu lead. Owned the 50-example dataset, six agent
  system prompts, demo direction, README, pitch.
- **[Member 2]** — Mobile lead. Built the Flutter app, agent cards, voice UI,
  Pricing Card, lifecycle screens, dispute UI, signed APK.
- **[Member 3]** — Backend Agents lead. Built Intent, Discovery, Ranking,
  Pricing agents and the eval harness.
- **[Member 4]** — Backend Workflows lead. Built Booking, Follow-up & Dispute,
  the orchestrator, Firestore persistence, receipt PDF.
- **[Member 5]** — Infra & Demo lead. Cloud Run, deployment pipeline, daily
  Antigravity trace capture, demo video edit, submission.


# ============================================================
# Section 11 — Acknowledgements
# ============================================================

## Acknowledgements

- **AI Seekho** and **InnoVista** for organising AI Seekho 2026.
- **MoITT** and **Telenor Pakistan** for sponsoring the hackathon.
- **Google for Developers** for Antigravity and Gemini API credits.
```

APPENDIX E

Pitch script — regional finals + Islamabad finals.

Three-minute pitch + five-minute Q&A. Read this aloud at least five times before each finals appearance. The phrases in **Urdu** are sacred — they are what differentiate this team. Numbers are filled in once you have actual Antigravity trace counts (Plan Artifacts, agent runs) from Infra.

```
# Bulao Pitch Script — Regional Finals (May 25-26) + Islamabad Finals (June 7)
# Target: 3-minute pitch + 5-minute Q&A from judges.

## PART 1 — Hook (0:00-1:00, ~150 words)

[Stand center stage. Phone in hand, screen off. Pause for 2 seconds.]

"Islamabad. Mid-May. The AC stops working. You open WhatsApp. You forward
the same message to four group chats: 'Koi achha AC technician ka number
bhejo G-13 ke liye?' Three hours later you have six numbers — half of them
old, half of them don't pick up. The one who does asks PKR 4,000 for a
visit. Your neighbour swears the same person charged him 1,500 last month.

This is how Pakistan's informal service economy works today. Through
WhatsApp forwards. Through trust networks that don't scale. Through prices
that nobody agrees on.

[Hold up the demo phone, screen on showing Bulao home screen.]

We built Bulao. You press a button. You speak in Urdu. Ninety seconds


later you have a verified provider, a transparent price, and a confirmed
booking. Six AI agents — visible on screen — do the work.

Bolo, aur kaam ho jaye."

[Pause for 2 seconds. Tap the mic button to start the demo video.]


## PART 2 — Live demo (1:00-3:30, ~200 words)

[Demo video plays muted on the projector behind you. You narrate over it.]

"Pehla agent samajhta hai kya kaam hai, kab chahiye, kitna complex hai.
It extracts seven fields from the Urdu sentence, with a confidence score.

[Beat at 1:00] Doosra agent paas ke providers dhoondhta hai. It uses a
30-minute travel buffer and suggests alternate slots if not enough match.

[Beat at 1:30] Yeh teesra agent — yeh sab se important hai. It scores
candidates across SIX factors — distance, availability, rating, on-time,
```

```
specialisation, risk — and explains its choice in natural Urdu with
specific numbers. Not 'best for you' — 'Ali sab se behtar — 1.4 kilometre
door, 4.8 rating, 98 percent on time'.

[Beat at 2:00] Chautha agent — pricing. Every rupee accounted for. Visit
fee, service time, complexity adjustment for inverter ACs, surge because
demand is high, welcome discount. AND a fairness note: 'Provider receives
1,920 after platform fee.'

[Beat at 2:30] Booking confirm. Receipt generated. Reminder scheduled.

[Beat at 3:20] AND — when something goes wrong — the sixth agent classifies
the dispute and proposes a fair resolution. Two stars? Partial refund OR
free re-service. The user chooses. Insaaf."

[Demo ends at 3:30. Step toward audience.]


## PART 3 — Antigravity (3:30-4:30, ~120 words)


"We built every line of this in five days, with Google Antigravity as our
orchestrator. Not as an autocomplete — as a colleague.

We ran <NUMBER> Plan Artifacts. We executed <NUMBER> agent runs across
five Manager View workspaces in parallel. We delegated browser
verification, eval harnessing, data generation, prompt iteration — and we
spent our human time on the things humans should — Urdu naturalness,
demo direction, customer empathy.

[Specific moment.] On Day 3, Antigravity reviewed our Ranking Agent's
output and flagged that our 'tradeoffs' field was being padded with English
idioms — 'cheaper but slower' instead of 'lekin Rashid 30 minute door hai'.
We iterated. Urdu naturalness went from 3.8 to 4.6 out of 5.

Yeh hai Antigravity-powered development."


## PART 4 — Why this wins (4:30-5:00, ~80 words)

"Two highest-weighted rubric criteria. Matching quality, 25 percent —
```

```
six explicit factors with weights per urgency level. Antigravity
integration, 20 percent — sixty Plan Artifacts in our trace deliverable.

Bulao is honest. The agents are visible. The prices are transparent.
The disputes are resolved fairly. The Urdu sounds like Urdu.

[Pause. Look directly at judges.]

Bolo, aur kaam ho jaye."

[Step back. Hand the mic.]


## Q&A — Anticipated questions

Q1: "What about real Maps integration?"
A: "Stretch goal. We chose mock data for demo reliability — Maps Places
calls cost real money and rate-limit. The mock has 100 providers across
14 neighborhoods. The Discovery Agent is API-agnostic; swapping in Places
is a one-day job."


Q2: "What about provider onboarding?"
A: "Roadmap. The dispute mechanism actually makes provider onboarding
EASIER — providers know that disputes are classified fairly, not on the
customer's word alone. Lower acquisition cost than apps that side with
the customer."

Q3: "Why six agents specifically?"
A: "Each agent maps to a distinct rubric requirement. Pricing maps to
'dynamic pricing'. Follow-up & Dispute maps to the 15-percent
dispute-and-reliability criterion. Five agents would have left points
on the table."

Q4: "Hardest part?"
A: "Urdu naturalness in the Ranking Agent. The model defaults to
English-translated Urdu. We built a quality sweep on Day 4 — three
inputs, five agents, fifteen graded outputs — and iterated until the
average grade hit 4.5 out of 5. That took six hours of prompt iteration
across two of our teammates."


Q5: "Cost to run?"
A: "Per booking: roughly PKR 4 in Gemini calls, free Cloud Run within
free tier, free Firestore within free tier. At 1,000 bookings/day, monthly
cost under USD 500."
```

APPENDIX F

Urdu phrasebook — the tone and terminology to enforce.

When you review Antigravity's Urdu output, you grade it against this phrasebook. Pakistani casual register, not formal Urdu, not Hindi-leaning vocabulary. This is what differentiates a 5/5 grade from a 3/5 grade.

```
Key terms and phrases — keep tone CASUAL, not formal.

GENERAL VERBS
  chahiye          = need / want         "AC chahiye"
  karwana          = get done            "theek karwana" = get repaired
  bhejo / bhejein  = send                "plumber bhejo"
  theek karna      = to fix              (formal)  -> use "theek karwana"
  pohanchna        = to reach            "pohanch jayenge" = will reach
  ho gaya          = it's done           "confirm ho gaya"

TIME
  abhi             = right now
  jaldi se         = quickly
  kal              = tomorrow (or yesterday — context tells you)
  kal subah        = tomorrow morning
  kal sham         = tomorrow evening
  parson           = day after tomorrow
  kabhi bhi        = whenever / flexible

LOCATION
  ghar pe / ghar mein  = at home
  mere ilaake mein     = in my area
  paas mein            = nearby
  fasla                = distance (use this, NOT "distance")

PRICE
  qeemat / paisay  = price / money
  rupay            = rupees                  (always PKR; never "Rs." in voiceovers)
  mehnga           = expensive
  sasta            = cheap
  thora            = a little               "thora surge"
  insaaf           = fair / fairness         (use in dispute messages)
```

```
QUALITY / FEELINGS
  achha            = good
  behtar           = better
  sab se behtar    = the best
  maafi mangte hain = we apologize
  shukriya         = thank you
  reservation     -> avoid (anglicism)        use "booking"


WORDS TO AVOID
  vidyut, jal      -> too Hindi-Sanskrit; use "bijli", "paani"
  utkrisht         -> too formal; use "behtar"
  pradhan          -> use first name
  kripya           -> use "please" or just imperative


PAKISTANI VS INDIAN ROMAN URDU
  Pakistan:  karwana, chahiye, ghar, kareingay
  India:     karvaana, chaahiye, ghara, karenge
  Pick Pakistani spellings throughout.


CODE-SWITCH PATTERNS (totally natural; do not avoid)
  "Kal morning AC service chahiye"
  "Booking confirm ho gaya"
  "30 minute mein pohanch jayenge"
  "Service fee 1,500 rupay"
  "Refund 3-5 din mein process hoga"


TONE PRINCIPLES

1. Speak the way a WhatsApp message from a friend sounds.
2. Never use "dear customer" or "esteemed user" — corporate, kill it.
3. Acknowledge feelings before facts ("Maafi mangte hain ke service achi nahi rahi").
4. Specific numbers feel honest. Round numbers feel evasive. "1,250 rupay" beats "approximately
PKR 1,300".
5. Tagline is sacred: "Bolo, aur kaam ho jaye." Never paraphrase it.
```